

Technical document

Automatic Cat Feeder

Ferran Vila Ordeig, Biel Tomàs Rifà, Pol Roch Jose, Georgina Garcia Vilaseca

Degree in Mechatronics Engineering

Subject: Integrated Projects II

Vic, June of 2026

Summary

Title: Automatic Cat Feeder

Authors: Ferran Vila Ordeig, Biel Tomàs Rifà, Pol Roch Jose, Georgina Garcia Vilaseca

Subject: Integrated Projects II

Date: 8 of June of 2026

Keywords: automatic feeder, mechatronics, pet care, embedded system, low-cost prototype

This project consists of the design and development of an automatic cat feeder intended to dispense controlled portions of dry food at scheduled times. The main purpose of the system is to improve the feeding routine of domestic cats by offering a reliable, programmable and low-cost solution for pet owners who cannot always be present at home.

Contents

Summary	i
List of Tables	iv
List of Figures	v
1 Introduction / State of the Art – Benchmarking	1
1.1 Background and Motivation	1
1.2 Why This Work Is Interesting	1
1.3 Previous Works and Their Limitations	2
1.3.1 Automatic and Smart Pet Feeders	2
1.3.2 Gravimetric Dispensing with Load Cells and the HX711	2
1.3.3 ESP32-Based IoT Systems for Embedded Animal Care	4
1.3.4 Feline Nutrition, Portion Control, and AI-Assisted Dietary Monitoring	5
1.3.5 Cloud Backends and Web Dashboards for IoT Devices	6
1.4 Summary of Gaps and Project Contribution	6

2	Objectives	7
2.1	Main Objective	7
2.2	Specific Objectives	7
2.2.1	SO1 – Gravimetric Portion Control	7
2.2.2	SO2 – Schedule-Based Autonomous Operation	7
2.2.3	SO3 – Remote Configuration and Monitoring	7
2.2.4	SO4 – Physical Touchscreen Interface	8
2.2.5	SO5 – Environmental and Operational Telemetry	8
2.2.6	SO6 – AI-Assisted Feeding Advisor	8
2.2.7	SO7 – Modular and Non-Blocking Firmware Architecture	8
2.2.8	SO8 – Functional Validation	8
3	Development of the Project	8
3.1	Analysis of the Problem and Possible Solutions	9
3.2	Description of the Proposed Design	10
3.3	Mechanical and Electrical Design Calculations	11
3.3.1	Dry Food Autonomy	11
3.3.2	Motor Torque Calculation	12
3.3.3	Power Consumption Estimation	14
3.4	Mechanical Design	15
3.5	Electrical Design	17
3.5.1	Power Architecture and Voltage Regulation	17
3.5.2	Microcontroller Pin Mapping	18
3.5.3	Subsystem Interfacing and Instrumentation	18
3.6	Control Strategy and Software	19

3.6.1	Firmware Structure	19
3.6.2	GPIO Assignment	20
3.6.3	System State Machine	20
3.6.4	Gravimetric Dispensing Algorithm	22
3.6.5	Display and Local User Interface	23
3.6.6	Cloud Integration and Companion Software	24
3.6.7	Safety and Error Handling	25
3.6.8	Libraries Used	25
3.7	Functional Testing of the Prototype	26
4	Cost of the Project	27
5	Conclusions	30
6	Bibliography	31
	Declaration of AI Use	33
A	Project Code Annexes	34
B	WebApp Front-End and Back-End ScreenShots	66
C	Electrical Schematic	69
D	Dimensioned 3D Part Drawings	70

List of Tables

List of Tables

1	Benchmarking of representative commercial smart feeders.	3
2	Comparison of filtering strategies for load-cell measurements.	4
3	Typical MER multipliers for cats by life stage or condition.	5
4	Comparison of possible dispensing solutions	10
5	Main technical characteristics of the automatic cat feeder prototype	11
6	Dry food autonomy calculation	12
7	Parameters used for the Janssen model	13
8	Motor torque calculation summary	14
9	Estimated power consumption of the automatic cat feeder	15
10	Estimated current consumption during dispensing operation	15
11	Main mechanical components of the prototype.	16
12	Main electronic components of the prototype.	17
13	Firmware file structure.	19
14	GPIO pin assignment of the production firmware.	20
15	System state machine.	21
16	Supabase tables used by the firmware.	24
17	Libraries used by the control software.	26
18	Planned functional tests for the automatic cat feeder	27
19	Estimated man-hours dedicated to the project.	28
20	Estimated man-hours per team member.	28
21	Material cost of the automatic cat feeder prototype.	29

22	Total project budget.	29
----	-------------------------------	----

List of Figures

List of Figures

1	Firmware state machine of the automatic cat feeder.	22
2	CatFeeder-T5 application capture 1.	66
3	CatFeeder-T5 application capture 2.	66
4	CatFeeder-T5 application capture 3.	67
5	CatFeeder-T5 application capture 4.	67
6	CatFeeder-T5 application capture 5.	68
7	CatFeeder-T5 application capture 6.	68
8	Electrical schematic of the automatic cat feeder prototype.	69

1 Introduction / State of the Art – Benchmarking

1.1 Background and Motivation

Pet ownership in Spain has grown substantially over the past decade. According to sector data published by ANFAAC, the country currently has more than 20 million registered companion animals [7]. A significant portion of these are domestic cats, and their owners increasingly face the same scheduling pressures that make consistent daily care difficult: long working hours, irregular routines, and short absences from home.

Feeding is one of the most critical routines in cat care, not because it is technically complex, but because it must happen at consistent times and in accurate quantities. Irregular meal timing causes anticipatory anxiety and food-seeking behaviour in cats [49]. Chronic overfeeding contributes to obesity, which is directly associated with diabetes mellitus, hepatic lipidosis, osteoarthritis, and feline lower urinary tract disease (FLUTD) [2]. The World Small Animal Veterinary Association (WSAVA), the American Animal Hospital Association (AAHA), and the American Association of Feline Practitioners (AAFP) discourage free-feeding and recommend portion control based on caloric calculations tied to each animal’s body condition score and metabolic rate [50] [52].

The practical problem is straightforward: most cat owners cannot be home for every meal, every day. Automatic feeders exist to fill this gap. The engineering problem, however, is that many available commercial and academic solutions fail to meet basic requirements for portion accuracy and reliable remote control. This project proposes a prototype that addresses these failures by combining closed-loop gravimetric dispensing, offline scheduling redundancy, a cloud backend, and an AI-assisted health monitoring layer.

1.2 Why This Work Is Interesting

The project is worth doing for two main reasons. The first is practical: a feeder that dispenses the wrong amount of food, or stops working when the Wi-Fi connection drops, is not just inconvenient; it can directly harm the animal it is supposed to care for. Solving this requires storing dry kibble without jamming, dispensing it in gram-accurate quantities regardless of kibble geometry, maintaining scheduled operation during network outages, and giving the owner meaningful remote visibility. These problems are manageable in isolation, but combining them in a single low-cost embedded device requires careful architectural choices.

The second reason is technical scope. The project integrates embedded firmware, motor control, sensor calibration, cloud connectivity, a reactive web dashboard, and a conversational AI agent. The choices made at each layer, from the stepper motor driver to the database schema, interact with each other. Managing those interactions is the core engineering challenge.

1.3 Previous Works and Their Limitations

1.3.1 Automatic and Smart Pet Feeders

Academic work on automatic pet feeders shows a clear progression. Early prototypes used 8-bit microcontrollers such as PIC, Arduino Uno, or Arduino Mega 2560, together with a DS3231 real-time clock module to trigger a small servo motor at scheduled intervals [1]. The servo either rotated a paddle or opened a trapdoor for a fixed duration. Some designs added an HC-SR04 ultrasonic sensor to estimate hopper food level, and others explored solar-powered variants to reduce grid dependency [1] [2].

The main limitation of these early designs is open-loop volumetric dispensing: the controller runs the motor for a set time and assumes that a consistent volume, and therefore a consistent weight, is released. This assumption fails in practice because dry kibble settles in the hopper over time, changing its bulk density, and different kibble shapes and sizes flow differently. A rotation that releases 20 g one morning may release 14 g the next [12]. No prototype in this generation included a feedback mechanism to verify the dispensed quantity.

As Wi-Fi-capable systems-on-chip became widely available, the ESP8266 and later the ESP32 replaced earlier microcontrollers in research prototypes, enabling local web interfaces and basic smartphone integration [4] [5]. These works improved scheduling flexibility, but the dispensing mechanism remained volumetric in nearly all cases [3]. Commercial products offer polished mobile applications, push notifications, and in some cases camera monitoring, but they present similar limitations.

The Petlibro AF203 is particularly instructive because its manufacturer acknowledges that portion weight can vary by up to 4 g on a standard 10 g serving [12]. For a cat on a strict dietary protocol, this is a meaningful error. The absence of a load cell in these products is a cost decision, but it means that strict caloric management is effectively impossible with current commercial feeders. Beyond portion accuracy, many commercial feeders operate within proprietary ecosystems with closed APIs and cloud services that may be discontinued.

1.3.2 Gravimetric Dispensing with Load Cells and the HX711

The solution to open-loop volumetric inaccuracy is a closed-loop gravimetric feedback system. In embedded applications, this usually means a strain-gauge load cell connected to an HX711 24-bit analog-to-digital converter. A single-point load cell converts mechanical deformation into a resistance change: when food lands in the bowl, the aluminium beam deflects, four strain gauges arranged as a Wheatstone bridge produce a small differential voltage, and the HX711 amplifies and converts that signal to a digital value at either 10 Hz or 80 Hz [7] [16].

In practice, this signal is noisy. Stepper motor coils generate electromagnetic interference, hard kibble impacting a metal bowl produces mechanical transients, and long-term sensor creep shifts the zero-point offset. Reliable gravimetric measurement in a mechatronic environment therefore requires calibration and filtering [6]. Calibration maps raw ADC counts to known reference weights

Table 1: Benchmarking of representative commercial smart feeders.

Brand and model		Dispensing	Connectivity	Notable feature	Key limitation
Petlibro AF203	Granary	Volumetric silicone rotor	Wi-Fi, proprietary app	Built-in camera and infrared level sensor	No load cell; up to ± 4 g error on a 10 g portion [12]
PETKIT Fresh Element		Volumetric / mixed	Wi-Fi, proprietary app	Dual hopper for food mixing	Closed ecosystem; no third-party smart home integration [8]
SureFeed Microchip		Motorised lid	RFID / Wi-Fi hub	Per-cat access control via microchip	Does not dispense from a hopper; only controls bowl access [15]
PetSafe Smart Feed		Volumetric conveyor belt	Wi-Fi, proprietary app	Slow-feed mode over 15 minutes	Subscription fees for advanced features; jams with large kibble [8]
Xiaomi Feeder	Smart	Volumetric rotor	Wi-Fi, Mi Home	Smart home integration	Closed API; dependent on regional cloud servers [8]

through a linear regression that yields a scale factor and zero offset. Auto-taring periodically recalculates the offset while the load cell is unloaded, compensating for thermal and structural drift [7].

Table 2: Comparison of filtering strategies for load-cell measurements.

Filter type	Mechanism	Suitable for	Limitation
Simple moving average	Averages the last N readings	Low-noise environments	Phase delay; vulnerable to impact spikes [17]
Median filter	Returns the middle value of a sorted window	Impulse noise rejection	Higher RAM and CPU cost than SMA [16]
IIR filter	Weighted sum of current inputs and past outputs	Steady-state resonance suppression	Can become unstable if coefficients are poorly designed [7]
Kalman filter	Recursive prediction-correction based on measurement variance	High-vibration environments	Requires matrix operations; computationally expensive [23]

For the proposed feeder, the chosen approach is a median filter applied to a rolling window of HX711 samples, combined with hardware capacitors on the ADC input pins for baseline electromagnetic interference rejection. The measured weight drives the dispensing control loop. As the weight approaches the target, the NEMA 17 stepper motor, driven by a DRV8825 driver, decelerates to prevent overshoot. The NEMA 17 was selected over servo motors and smaller gear steppers because its holding torque of approximately 40–60 Ncm is sufficient to force past kibble jams without stalling [24]. The DRV8825 supports up to 1/32 microstepping, reducing mechanical resonance and acoustic noise, while the AccelStepper library provides non-blocking trapezoidal acceleration profiles [27] [28].

1.3.3 ESP32-Based IoT Systems for Embedded Animal Care

The ESP32 WROOM-32 was chosen as the microcontroller because of its native Wi-Fi and BLE support, dual-core Xtensa LX6 architecture, and clock frequency of up to 240 MHz. Under FreeRTOS, motor control and network communication can run on separate cores [5]. This separation matters because TLS handshakes or large JSON payloads can take tens of milliseconds. On a single execution thread, such operations can stall the motor control loop and cause the stepper to miss steps or jitter. Assigning the AccelStepper loop and HX711 polling to Core 1, while network operations run on Core 0, avoids this interference [31] [32].

The firmware also avoids blocking delays. The standard Arduino `delay()` function suspends execution, which can break network connections, miss hardware interrupts, and prevent watchdog refreshes. Production-grade embedded IoT systems therefore use `millis()`-based scheduling and non-blocking state machines [30] [36]. The feeder firmware transitions through defined states such as IDLE, TARRING, DISPENSING, PAUSED, and ERROR using elapsed-time comparisons rather than blocking delays.

For cloud communication, the firmware polls the Supabase REST API over HTTPS using zero-buffer stream parsing. Instead of loading the full HTTP response into heap memory, which risks fragmentation and crashes on a device with limited RAM, the parser reads the TCP stream byte by byte and extracts the required keys directly [40]. Offline reliability is provided by a DS3231 RTC module with a temperature-compensated crystal oscillator and backup battery. If Wi-Fi is lost, the firmware falls back to DS3231 timestamps, continues executing scheduled feedings, and queues events for later cloud synchronisation [1].

The 3.5-inch ILI9488 touchscreen with XPT2046 controller connects over SPI. This combination has a known hardware issue: the ILI9488 does not tri-state its MISO line when chip select is deasserted, preventing the ESP32 from communicating with the XPT2046 on the same SPI bus [43] [44]. The firmware resolves this through careful SPI transaction management using the TFT_eSPI library and by assigning the XPT2046 chip select to GPIO 5 rather than GPIO 21, which would conflict with the DS3231 I2C SDA line. A DHT22 sensor monitors humidity and temperature inside the hopper because excessive humidity can accelerate oxidative rancidity and mould growth in stored kibble [52].

1.3.4 Feline Nutrition, Portion Control, and AI-Assisted Dietary Monitoring

Veterinary guidelines from WSAVA and AAHA/AAFP frame portion accuracy as a medical requirement, not just a convenience feature. The standard caloric calculation begins with the Resting Energy Requirement (RER):

$$\text{RER} = 70 \times W_{kg}^{0.75} \quad (1)$$

where W_{kg} is the cat’s body weight in kilograms. The Maintenance Energy Requirement (MER) is then calculated by multiplying the RER by a life-stage factor [53] [54] [55].

Table 3: Typical MER multipliers for cats by life stage or condition.

Life stage / condition	MER multiplier	Notes
Healthy, intact adult	$1.4\text{--}1.6 \times \text{RER}$	Baseline for active, non-neutered cats
Neutered adult	$1.2 \times \text{RER}$	Neutering reduces metabolic rate and requires caloric reduction
Inactive or obesity-prone	$1.0 \times \text{RER}$	For sedentary indoor cats maintaining current weight
Weight loss protocol	$0.8 \times \text{RER}$ of ideal weight	Applied to ideal weight, not current weight [56]
Kittens under 4 months	$2.5\text{--}3.0 \times \text{RER}$	High requirement for skeletal and neurological growth

Translating caloric targets into physical food portions is complicated by the wide variation in commercial kibble caloric density, which may range from 300 to 500 kcal per cup depending on brand and formula [57]. A gravimetric feeder bypasses this ambiguity by dispensing a specific gram weight and stopping when the target is reached, regardless of how the kibble is packed in the hopper.

Beyond static portion control, recent research has begun applying anomaly detection to feeding behaviour logs. If a cat consistently takes longer to finish its portion or leaves a larger fraction uneaten over several days, these trends can indicate dental problems, stomatitis, or early renal disease before clinical symptoms appear [10]. The VetGat AI chatbot implemented in this project ingests time-series feeding data from the Supabase backend and uses Retrieval-Augmented Generation (RAG) to ground its dietary recommendations in verified WSAVA and AAHA guidelines rather than generating responses from the language model alone [58] [69].

1.3.5 Cloud Backends and Web Dashboards for IoT Devices

Supabase was selected as the cloud backend over alternatives such as Firebase because it is built on PostgreSQL, which is well suited for structured relational telemetry data [39] [60]. For each database table, Supabase automatically generates a REST API through the PostgREST layer, allowing the ESP32 to execute CRUD operations over HTTPS without custom middleware [61] [62].

Row-Level Security policies enforced natively within Postgres ensure that each device can only read its own configuration commands and write to its own telemetry rows [63]. Supabase’s Realtime API streams database write-ahead-log changes to connected clients through WebSockets, allowing the React/Vite dashboard to update live when the ESP32 posts a new feeding event [64]. Edge Functions, implemented as serverless scripts in a Deno runtime, handle asynchronous backend logic such as push notifications when humidity thresholds are exceeded [39].

The web dashboard is structured as a React Single Page Application built with Vite. It provides six functional sections: dashboard overview, cat profiles, feeding schedules, history and analytics, device management, and calibration settings. The VetGat chatbot is orchestrated using n8n following the RAG pattern: veterinary documentation is chunked, converted to vector embeddings, and stored in Supabase through the pgvector extension. When a user submits a query, n8n retrieves the most relevant documentation chunks by cosine similarity, queries the live feeding database, and passes both to the language model to generate a grounded, data-specific response [69] [71] [72].

1.4 Summary of Gaps and Project Contribution

The problems are consistent across the literature. Commercial feeders accept large portion errors because adding a load cell increases unit cost, and their cloud ecosystems are often proprietary and fragile. Academic prototypes provide better architectural transparency but rarely combine accurate sensing, cloud connectivity, and intelligent monitoring in a single coherent system.

This project addresses these gaps directly. Closed-loop gravimetric control with an HX711 and a microstepped NEMA 17 replaces open-loop volumetric dispensing. FreeRTOS dual-core task segregation ensures that the motor control loop is never blocked by network operations. DS3231 hardware backup makes the device reliable when Wi-Fi is unavailable. Supabase provides an open, self-hostable backend without proprietary lock-in. Finally, the VetGat RAG agent turns feeding telemetry into actionable dietary advice grounded in WSAVA and AAHA clinical guidelines.

2 Objectives

This section defines what the CatFeeder-T5 is expected to achieve by the end of the project. The objectives were written after reviewing existing feeders and analysing what is technically feasible within the scope of Integrated Project II at UVic-UCC.

2.1 Main Objective

Design and build a gravimetric automatic cat feeder controlled by an ESP32 microcontroller, capable of dispensing precise dry food portions on a schedule defined by the user, either locally or remotely, while logging every feeding event to a cloud database and providing both a physical touchscreen interface and a web dashboard for monitoring and control.

2.2 Specific Objectives

2.2.1 SO1 – Gravimetric Portion Control

Dispense food portions with an error below ± 5 g per cycle. The dispensing mechanism, based on a NEMA 17 stepper motor driven through a DRV8825 driver, operates in a closed loop with the HX711 load cell: the motor runs until the measured weight reaches the target value and then stops. Volume-based estimation is explicitly avoided.

2.2.2 SO2 – Schedule-Based Autonomous Operation

Execute feeding schedules without requiring any user action at the time of feeding. Schedules are stored both in the DS3231 RTC, for offline operation during network outages, and in Supabase, as the primary remote configuration source. If Wi-Fi is unavailable, the RTC-backed schedule continues to run independently.

2.2.3 SO3 – Remote Configuration and Monitoring

Allow users to define and modify feeding schedules, portion sizes, and device settings from a React/Vite web dashboard without physical access to the device. The ESP32 polls Supabase through REST at regular intervals to fetch new commands and push telemetry data, including weight, temperature, humidity, and feed events.

2.2.4 SO4 – Physical Touchscreen Interface

Provide a standalone user interface on a 3.5-inch ILI9488 touchscreen that allows manual food dispensing, mode switching, and basic status reading without any network connection. The interface includes an idle screensaver and a four-corner unlock pattern to prevent accidental activation by the cat.

2.2.5 SO5 – Environmental and Operational Telemetry

Record and transmit temperature readings from the DHT22 sensor and humidity readings alongside each feeding event. This data is stored in Supabase and made accessible from the dashboard, providing context for detecting anomalies in food storage conditions.

2.2.6 SO6 – AI-Assisted Feeding Advisor

Integrate an n8n webhook-based chatbot, named VetGat, that queries the Supabase feeding history and provides personalised nutrition and health observations. The chatbot should detect patterns such as missed meals, consistent under-eating, or critically low hopper weight, and report them proactively.

2.2.7 SO7 – Modular and Non-Blocking Firmware Architecture

Implement the ESP32 firmware as a set of independent modules, including motor, scale, RTC, network, display, and sensors, coordinated by a finite state machine. The main loop must never block: all timing uses `millis()`-based intervals, and `AccelStepper::run()` is called on every iteration to maintain smooth motor motion regardless of network or sensor activity.

2.2.8 SO8 – Functional Validation

Test each subsystem individually before integration and validate the complete system against three core metrics: portion accuracy, with a target error below ± 5 g; schedule execution reliability, with 100% of scheduled feeds executed within ± 30 s; and system uptime over a continuous 48-hour run.

3 Development of the Project

This chapter describes the technical development of the automatic cat feeder prototype. It includes the analysis of possible solutions, the proposed design, the mechanical and electrical architecture, the control strategy, the main calculations, and the validation tests planned for the prototype. The

objective of this section is to justify the design decisions and demonstrate that the final solution is coherent with the project objectives.

3.1 Analysis of the Problem and Possible Solutions

The main technical problem of the project is to design a system capable of dispensing dry cat food automatically, reliably, and with controlled portions. To achieve this, several possible solutions were analysed before selecting the final design.

The first possible solution consisted of using a simple gravity-based dispenser controlled by a servo motor. In this design, a hatch or gate opens for a predefined amount of time, allowing food to fall into the bowl. This solution is simple, low-cost, and easy to implement. However, it has an important limitation: the quantity of dispensed food depends on the size, shape, density, and flow behaviour of the dry food. As a result, the accuracy of the portions may vary significantly.

A second possible solution was based on a rotating wheel or compartment-based mechanism. This approach divides food into predefined cavities and releases one or more portions when the mechanism rotates. It provides better repeatability than a simple gate system, but it requires careful mechanical design and may be less adaptable to different portion sizes or food types.

The third solution consisted of using an auger screw mechanism driven by a stepper motor. In this configuration, the motor rotates a screw that moves food from the storage container to the output area. This solution provides better control because the amount of food dispensed can be related to the number of motor steps. It also allows smoother and more progressive dispensing.

A fourth solution was considered by combining the auger screw mechanism with a load cell. In this configuration, the stepper motor controls the movement of the food, while the load cell measures the amount dispensed in real time. This allows the system to stop the motor once the target portion has been reached, improving portion accuracy and reducing the risk of overfeeding or underfeeding.

The advantages and limitations of the different dispensing alternatives are compared in Table 4. Based on this analysis, the selected solution for the prototype was an auger screw mechanism driven by a stepper motor and complemented with a load cell for feedback control. This option was considered the most suitable because it combines controlled dispensing with real-time portion measurement, improving the accuracy and reliability of the system.

Table 4: Comparison of possible dispensing solutions

Solution	Advantages	Limitations	Suitability
Gravity gate with servomotor	Simple, cheap, low power	Poor portion accuracy	Medium
Rotating wheel	Repeatable portions, compact	Less flexible	Medium-high
Stepper motor with auger screw	Good control and precision	Requires motor driver	High
Stepper motor with auger screw and load cell	Better portion accuracy, feedback control	More complex electronics and calibration	Very high

3.2 Description of the Proposed Design

The proposed design is an automatic cat feeder with a vertical food storage container, a controlled dispensing mechanism, an electronic control unit, a local user interface, and a power supply system. The feeder is designed to dispense dry food at configurable times and with adjustable portion sizes.

The system is composed of the following main subsystems: the mechanical structure, the food container, the stepper motor dispensing mechanism, the ESP32-based electronic control unit, the load cell, the RTC module, the environmental sensor, the user interface, and the power supply system.

The food is stored in the upper container and moves by gravity towards the lower dispensing area. At this point, the auger screw, driven by the NEMA 17 stepper motor, transports the food towards the outlet. The number of motor steps defines the approximate amount of food moved, while the load cell provides feedback by measuring the real amount dispensed into the bowl. This combination allows the system to improve the repeatability and accuracy of each portion.

The ESP32 microcontroller acts as the main control unit. It manages the feeding schedule, controls the motor driver, reads the load cell through the HX711 amplifier, obtains time information from the RTC module, and can process data from the DHT22 temperature and humidity sensor. The ESP32 was selected because it provides enough processing capacity, multiple input/output pins, low power consumption, and wireless connectivity for possible future improvements.

The feeder also includes a local user interface for basic configuration and monitoring. This interface can be used to define feeding times, adjust portion sizes, and check the general status of the system. The final prototype is intended to be compact, low-cost, reliable, and easy to maintain.

The main technical characteristics of the automatic cat feeder are listed in Table 5.

Table 5: Main technical characteristics of the automatic cat feeder prototype

Characteristic	Description
Application	Automatic dry food dispenser for domestic cats
Food type	Dry cat food
Food capacity	Approximately 3 kg
Estimated autonomy	Approximately 45.6 days for one adult cat
Daily food quantity	Approximately 65.8 g/day
Dispensing mechanism	Auger screw mechanism
Actuator	NEMA 17 stepper motor
Motor control	Stepper motor driver
Control unit	ESP32 microcontroller
Portion control	Motor steps and load cell feedback
Weight sensor	Load cell with HX711 amplifier
Time management	RTC module
Environmental sensing	DHT22 temperature and humidity sensor
Power supply	12 V line for the motor and 5 V / 3.3 V line for electronics
Estimated energy consumption	0.73–2 Wh/day
Programming language	C/C++ using the Arduino framework
User interface	Local interface for configuration and system status
Degrees of freedom	1 rotational degree of freedom in the dispensing mechanism
Main materials	Plastic or 3D-printed parts for the structure and dispensing system

3.3 Mechanical and Electrical Design Calculations

This section presents the main calculations used to size the food storage capacity, estimate the required motor torque, and evaluate the power consumption of the automatic cat feeder. These calculations are necessary to justify the selection of the main components and verify that the design is technically feasible.

3.3.1 Dry Food Autonomy

The feeder is designed for an adult cat with an approximate weight between 4 kg and 5 kg. A high daily energy requirement of 250 kcal/day was considered in order to ensure that the system is sized for a demanding case. The selected dry food has an average energy density of 3.8 kcal/g.

The required amount of food per day is calculated as:

$$m_{day} = \frac{E_{day}}{E_{food}} \quad (2)$$

where:

$$E_{day} = 250 \text{ kcal/day} \quad (3)$$

$$E_{food} = 3.8 \text{ kcal/g} \quad (4)$$

Therefore:

$$m_{day} = \frac{250}{3.8} = 65.79 \text{ g/day} \quad (5)$$

Considering a total food capacity of 3000 g, the estimated autonomy of the feeder is:

$$D = \frac{m_{total}}{m_{day}} \quad (6)$$

$$D = \frac{3000}{65.79} = 45.6 \text{ days} \quad (7)$$

As a result, the feeder can theoretically operate for approximately 45 days before requiring a refill, assuming one adult cat and a constant daily consumption. The parameters used in this calculation are summarized in Table 3.

Table 6: Dry food autonomy calculation

Parameter	Value	Unit
Cat daily energy requirement	250	kcal/day
Dry food energy density	3.8	kcal/g
Required dry food per day	65.79	g/day
Total dry food capacity	3000	g
Estimated autonomy	45.6	days

3.3.2 Motor Torque Calculation

In order to estimate the force exerted by the dry food at the bottom of the storage container, the Janssen model was considered. This model is commonly used to estimate the pressure distribution in granular materials stored inside silos or containers.

Table 7: Parameters used for the Janssen model

Parameter	Description	Value	Unit
R	Container radius	0.09	m
ρ	Dry food density	500	kg/m ³
μ	Friction coefficient	0.4	-
K	Lateral pressure ratio	0.4	-

According to the Janssen model, and using the parameters listed in Table 7, the estimated pressure at the bottom of the container is:

$$\sigma = 783 \text{ Pa} \quad (8)$$

The force applied at the bottom surface is calculated using:

$$F = \sigma \cdot A \quad (9)$$

where the bottom area is:

$$A = \pi R^2 \quad (10)$$

$$A = \pi \cdot 0.09^2 = 0.0254 \text{ m}^2 \quad (11)$$

Therefore:

$$F = 783 \cdot 0.0254 = 19.9 \text{ N} \quad (12)$$

The torque required to move the dispensing screw is estimated from:

$$T = F \cdot r \quad (13)$$

where the screw radius is:

$$r = 0.01 \text{ m} \quad (14)$$

Thus:

$$T = 19.9 \cdot 0.01 = 0.199 \text{ N m} \quad (15)$$

The selected motor has a nominal torque of:

$$T_{motor} = 0.4 \text{ N m} \quad (16)$$

The safety factor is calculated as:

$$SF = \frac{T_{motor}}{T_{required}} \quad (17)$$

$$SF = \frac{0.4}{0.199} = 2.01 \quad (18)$$

Since the safety factor is approximately 2, the selected motor is considered suitable for the dispensing mechanism. This margin helps compensate for possible friction losses, irregular food flow, small blockages, or variations in the dry food geometry. The final results of the motor torque calculation are summarized in Table 8.

Table 8: Motor torque calculation summary

Parameter	Value	Unit
Estimated bottom pressure	783	Pa
Force at the bottom	19.9	N
Screw radius	0.01	m
Required torque	0.199	Nm
Selected motor torque	0.4	Nm
Safety factor	2.01	-

3.3.3 Power Consumption Estimation

The automatic cat feeder requires two different voltage levels: 12 V for the NEMA 17 stepper motor and 5 V / 3.3 V for the electronic control system. The main components considered in the power estimation are the stepper motor, the motor driver, the ESP32 microcontroller, the HX711 load cell amplifier, the DHT22 temperature and humidity sensor, and the RTC module.

The estimated voltage and consumption values of the main components are summarized in Table 6.

The feeder does not operate continuously at maximum power. The motor is only activated during the dispensing process, and the Wi-Fi connection of the ESP32 is only required at specific moments.

Table 9: Estimated power consumption of the automatic cat feeder

Component	Voltage	Estimated consumption
NEMA 17 stepper motor	12 V	8–20 W
Stepper motor driver	5 V	-
ESP32	5 V / 3.3 V	20–80 mA
HX711 amplifier	5 V	< 1.5 mA
DHT22 sensor	5 V	Low current consumption
RTC module	5 V	Low current consumption

Therefore, the average daily consumption is significantly lower than the theoretical maximum instantaneous consumption.

Considering limited motor operation and non-continuous Wi-Fi usage, the estimated current ranges are shown in Table 7.

Table 10: Estimated current consumption during dispensing operation

Voltage line	Estimated current
12 V line	0.7–1.7 A
5 V line	0.1–0.3 A

From these assumptions, the estimated daily energy consumption of the system is between:

$$E_{day} = 0.73 - 2 \text{ Wh/day} \quad (19)$$

This low energy demand is achieved by applying basic power-saving strategies. The motor driver should be disabled when the feeder is not dispensing food, reducing unnecessary current consumption. In addition, the Wi-Fi module should only be activated when communication is required, instead of keeping it permanently active at maximum power.

These design decisions improve the energy efficiency of the prototype and make it suitable for operation with a small external power supply or a battery-based system.

3.4 Mechanical Design

The mechanical architecture of the prototype is based on a modular assembly of 3D-printed components designed to support food storage, controlled dispensing, structural stability, and integration of the electronic subsystems. The design prioritizes ease of manufacturing, low production cost, and adaptability to different container sizes and dispensing mechanisms. The dimensioned drawings of the 3D-printed parts are included in Appendix D.

The main mechanical components used in the prototype are listed in Table 11.

Table 11: Main mechanical components of the prototype.

Component	Function	Justification
Main body	Structural frame of the feeder	Supports all mechanical and electronic subsystems
Food hopper	Dry food storage	Provides sufficient food capacity and controlled flow
Dispensing wheel / screw	Food dispensing mechanism	Enables controlled portion delivery
Bowl support	Bowl positioning and stabilization	Ensures accurate weight measurement and feeding area alignment
Motor mount	Stepper motor fixation	Maintains correct alignment between motor and dispensing mechanism
Electronics cover	Protection of electronic components	Prevents accidental contact and protects wiring

Structural Design and Material Selection All structural components are manufactured using fused deposition modeling (FDM) 3D printing technology. Polylactic Acid (PLA) is selected as the primary material due to its low cost, ease of printing, dimensional stability, and sufficient mechanical strength for indoor operation. Since the feeder is intended for domestic use and is not exposed to significant environmental stresses, PLA provides an adequate balance between performance and manufacturability.

The main body serves as the structural backbone of the system, supporting the food hopper, dispensing mechanism, electronics enclosure, and bowl support. The modular design allows individual components to be replaced or modified without redesigning the complete assembly.

Food Storage and Dispensing Mechanism The food hopper is designed to store dry cat food and guide it toward the dispensing mechanism using gravity. The hopper geometry minimizes food accumulation zones and promotes a consistent flow of kibble toward the outlet.

Portion control is achieved through a rotating dispensing wheel or screw mechanism driven by the NEMA 17 stepper motor. The geometry of the dispensing element determines the quantity of food delivered during each rotation. Because commercial cat food varies considerably in pellet size, shape, and surface friction, the dispensing mechanism must be experimentally validated using the specific food intended for operation. Variations in kibble dimensions may result in flow restrictions, inaccurate portion sizes, or mechanical jamming.

As part of the mechanical design process, detailed 3D models of all mechanical components were created using CAD software. These models were used to define the geometry, dimensions, and functional characteristics of each part, ensuring proper fit and interaction between components. The 3D designs also facilitated design verification, assembly analysis, and the generation of technical drawings for manufacturing and documentation purposes. The complete set of technical drawings and 3D CAD models is provided in the Annex, which contains the full mechanical documentation

generated throughout the project.

3.5 Electrical Design

The electrical architecture of the prototype is built around the ESP32 WROOM-32 microcontroller. This unit coordinates all actuators, sensors, timekeeping modules, and user interfaces. The system balances the high-current demands of the motor with the sensitive digital signals of the instrumentation sensors.

The main electronic components used in the prototype are listed in Table 12.

Table 12: Main electronic components of the prototype.

Component	Function	Justification
ESP32 WROOM-32	Main control unit	Integrated Wi-Fi, enough GPIO pins, low cost
NEMA 17 stepper motor	Dispensing actuator	Precise position and speed control
DRV8825 driver	Stepper motor control	High-current handling, microstepping capability
Load cell (1 kg) + HX711	Food weight measurement	Enables closed-loop portion control
DHT22	Environmental tracking	Monitors food storage conditions, temperature, and humidity
RTC DS3231	Hardware timekeeping	Maintains accurate schedules without network dependency
3.5-inch TFT touch display	Local user interface	Allows standalone local configuration
12 V UPS / battery system	Power supply	Enables autonomous operation during outages

3.5.1 Power Architecture and Voltage Regulation

The prototype is powered by a 12 V mini uninterruptible power supply (UPS) system. Because the chosen subsystems operate at three distinct voltage levels, 12 V, 5 V, and 3.3 V, a structured voltage regulation strategy is implemented to avoid logic instability and component damage.

- **12 V power rail:** Fed directly from the 12 V output of the UPS to power the NEMA 17 stepper motor through the DRV8825 driver hardware. This design isolates heavy inductive motor spikes from the sensitive microchip rails.
- **5 V power rail:** Supplied directly from the 5 V output port of the UPS to the USB interface of the ESP32 development board. This rail delivers stable power to the microcontroller.

- **3.3 V power rail:** Supplied by the onboard low-dropout (LDO) linear regulator of the ESP32. It provides clean logic power to the HX711 amplifier, DHT22 sensor, TFT screen and DS3231 real-time clock module.

All voltage rails share a common ground reference. This is required to ensure that the control signals sent from the ESP32 to the DRV8825 driver and the digital communication lines used by the sensors are correctly interpreted. Without a common ground, the system could present unstable behaviour, communication errors, or incorrect motor control.

3.5.2 Microcontroller Pin Mapping

The complete pin assignment mapping for the microcontroller is configured as shown in Table 14.

The complete electrical schematic of the prototype is shown in Appendix C, Figure 8.

3.5.3 Subsystem Interfacing and Instrumentation

Actuator Control: NEMA 17 and DRV8825 The NEMA 17 stepper motor requires a higher driving current than the microcontroller GPIO lines can supply directly. Therefore, a dedicated DRV8825 stepper motor driver is placed between the ESP32 and the motor coils.

The ESP32 sends hardware timing pulses to the STEP pin to control speed and angular displacement, while the state of the DIR pin determines the rotation direction. To minimize idle power consumption and eliminate motor heating during standby modes, the ENABLE pin is pulled high by software to completely disable the driver output stage when a feeding cycle is inactive.

A bulk capacitor should be placed close to the motor power input of the DRV8825 to reduce voltage spikes generated by the inductive load. In addition, the current limit of the driver must be adjusted according to the rated current of the selected NEMA 17 motor in order to avoid overheating or damaging the motor and the driver.

Closed-Loop Weight Sensing: Load Cell and HX711 To verify accurate portion delivery, a closed-loop measurement feedback mechanism is implemented using a strain-gauge load cell beneath the bowl. The continuous, minute analog voltage variations from the load cell are passed to an HX711 24-bit analog-to-digital converter (ADC). The HX711 filters and amplifies this signal, translating it into a calibrated digital weight metric readable by the ESP32 through a synchronized two-wire serial interface.

Hardware Real-Time Clock: DS3231 Relying solely on internal microchip software timers can lead to significant time drift over extended intervals. To prevent this, a dedicated DS3231 RTC module is attached through the hardware I^2C bus. Armed with an independent CR2032 backup

battery cell, this module maintains precise local timekeeping even during a complete breakdown of primary system power, ensuring that feeding schedules are retained without network dependency.

3.6 Control Strategy and Software

The control and software architecture of the automatic cat feeder is implemented in C++ using the Arduino IDE 2.x and the ESP32 board support package from Espressif. The firmware follows a multi-tab Arduino sketch structure: every `.ino` file in the `CatFeeder/` folder is compiled as part of the same translation unit, while each file keeps the logic of one subsystem separated from the rest of the program. The complete source code is therefore not reproduced in this report; the code annex is included in Appendix A.

The main design constraint of the firmware is non-blocking execution. The main `loop()` function does not use `delay()`. Timed operations are implemented with `millis()`-based state checks so that motor stepping, load-cell reading, display refreshing, button handling, environmental sensing, and network polling can run in the same execution cycle without one task freezing the others.

3.6.1 Firmware Structure

The production firmware is divided into the files shown in Table 13. This modular organisation makes the code easier to inspect and modify, because each peripheral or service is associated with a specific file.

Table 13: Firmware file structure.

File	Role
<code>CatFeeder.ino</code>	Main orchestrator. Contains <code>setup()</code> and <code>loop()</code> , manages the system state machine, and calls the update functions of each module.
<code>config.h</code>	Central configuration header. Defines GPIO assignments, Wi-Fi credentials, Supabase endpoint and keys, HX711 calibration factor, motor parameters, and feeding tolerances.
<code>Motor.ino</code>	Stepper motor control through the DRV8825 driver using the AccelStepper library. Implements gravimetric dispensing based on measured weight instead of fixed motor steps.
<code>Scale.ino</code>	HX711 interface. Handles raw readings, tare, moving-average filtering, and calibration against a known reference weight.
<code>Display.ino</code>	ILI9488 TFT display management through TFT_eSPI. Draws the information screens and handles the touch-based local interface.
<code>RTCManager.ino</code>	DS3231 real-time clock interface through I2C. Reads the current time, evaluates local schedules, and synchronises schedules after successful network polls.
<code>Environment.ino</code>	DHT22 temperature and humidity sensing. Updates environmental telemetry every 10 seconds.
<code>Inputs.ino</code>	Physical input management. Reads the manual feed, tare, menu, and presence inputs with 50 ms software debounce.
<code>Network.ino</code>	Wi-Fi and Supabase REST API communication. Polls commands, uploads feed events and telemetry, and registers device heartbeats.

3.6.2 GPIO Assignment

The GPIO map was defined to avoid conflicts between the SPI display, touch controller, I2C real-time clock, motor driver, sensors, and physical inputs. In earlier revisions, the touch chip-select signal conflicted with the DS3231 I2C data line. The corrected map assigns `TOUCH_CS` to GPIO 5 and keeps GPIO 21 for I2C SDA. GPIO 34 and GPIO 35 are input-only pins and do not include internal pull-up resistors, so they require external 10 k Ω pull-up resistors to 3.3 V.

Table 14: GPIO pin assignment of the production firmware.

Peripheral / signal	GPIO	Notes
TFT MOSI (SDI)	23	Shared SPI bus
TFT MISO (SDO)	19	Shared SPI bus
TFT SCLK	18	Shared SPI bus
TFT CS (ILI9488)	15	Display chip select
TFT DC	2	Data/command select
TFT RESET	4	Hardware reset
TFT LED backlight	32	Permanently enabled
TFT TOUCH_CS (XPT2046)	5	Touch chip select
DRV8825 ENABLE	25	Active LOW; disabled between dispenses
DRV8825 STEP	26	Step pulse output
DRV8825 DIR	27	Direction control
HX711 DOUT	16	Load-cell data
HX711 SCK	17	Load-cell clock
DHT22 DATA	33	Temperature and humidity
DS3231 SDA	21	I2C data
DS3231 SCL	22	I2C clock
BTN_FEED	13	Manual dispense button
BTN_TARE	14	Manual tare button
BTN_MENU	34	Screen rotation; external 10 k Ω pull-up
SQUISHY presence sensor	35	Cat presence input; external 10 k Ω pull-up

3.6.3 System State Machine

The firmware operates as a finite-state machine. The current state is stored in `CatFeeder.ino`, and transitions are triggered by sensor readings, local buttons, network commands, or timer events. The four main states are summarised in Table 15.

Table 15: System state machine.

State	Description	Entry conditions	Exit conditions
STATE_BOOT	Hardware initialisation of SPI, I2C, HX711, Wi-Fi, and display.	Power-on or reset.	All peripherals initialised → STATE_IDLE. Peripheral fault → remain in boot with error display.
STATE_IDLE	Normal standby. RTC is polled every second; display and network polling remain active.	Boot completed or dispense cycle finished.	Schedule reached, local button pressed, or remote command received → STATE_DISPENSING.
STATE_DISPENSING	Active food dispensing. Motor is enabled and the load cell is read continuously.	Schedule, manual command, or Supabase command.	Target reached → STATE_IDLE. Timeout exceeded → STATE_ERROR.
STATE_ERROR	Fault condition. Motor is stopped, driver is disabled, and the error is shown and logged.	Motor timeout, HX711 fault, or I2C bus error.	Manual reset or reboot → STATE_BOOT.

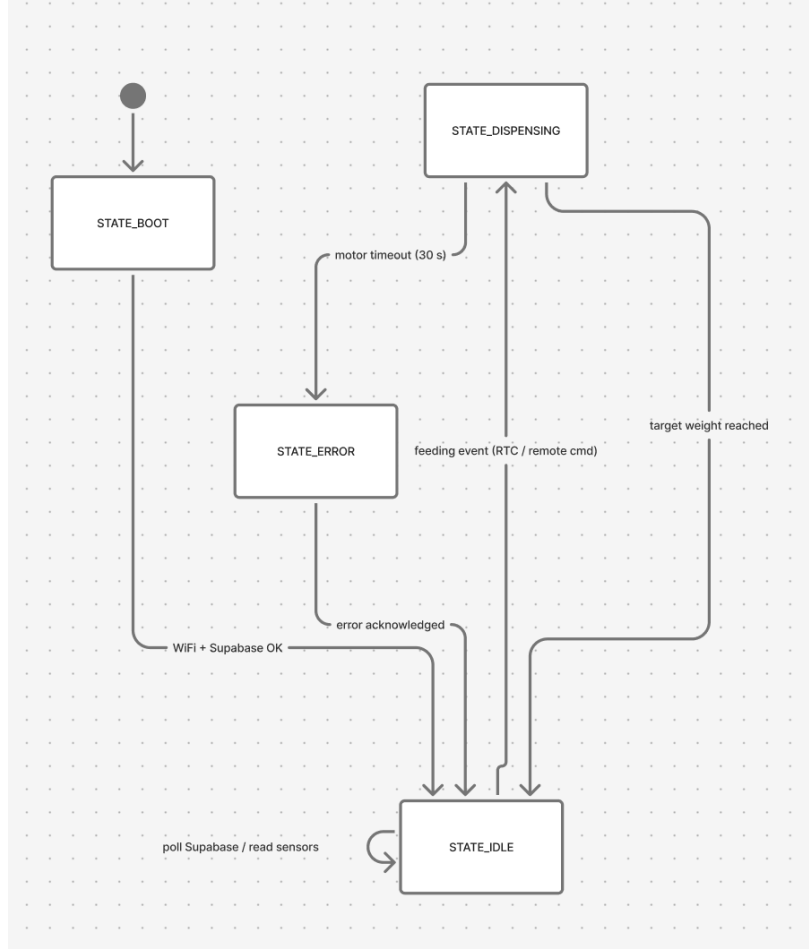


Figure 1: Firmware state machine of the automatic cat feeder.

3.6.4 Gravimetric Dispensing Algorithm

The main feeding algorithm is implemented in `Motor.ino`. Its purpose is to stop the auger when the measured food mass reaches the target portion. This approach compensates for variations in kibble geometry, bulk density, and auger filling, which cannot be corrected by a purely time-based or step-count-based method.

When `STATE_DISPENSING` starts, the firmware first reads the current scale value and stores it as the tare reference `weight_before_g`. The DRV8825 driver is then enabled by pulling GPIO 25 LOW, and the stepper starts generating pulses at the configured dispensing speed, with a default value of 400 steps/s. On every iteration of `loop()`, the current weight is read from the HX711 and the dispensed mass is calculated as:

$$m_{dispensed} = m_{current} - m_{before} \quad (20)$$

The motor is stopped when the measured mass reaches the target portion minus the configured tolerance:

$$m_{dispensed} \geq m_{target} - m_{tolerance} \quad (21)$$

The default tolerance is 2 g. Once the condition is satisfied, the motor stops immediately, the driver is disabled to avoid thermal buildup, and the feed event is recorded with actual weight, timestamp, schedule identifier, and final status. The HX711 calibration factor stored in `config.h` is approximately `-2025.0`; the sign is negative because the load cell is mounted in an inverted orientation, not because of a software error.

A timeout check runs in parallel with the dispensing loop. If the motor remains in `STATE_DISPENSING` for more than `STEPPER_MAX_RUN_MS`, set to 30000 ms, without reaching the target weight, the firmware assumes a blockage or sensor fault. It stops the motor, disables the driver, enters `STATE_ERROR`, and logs the event with the status `motor_timeout`.

3.6.5 Display and Local User Interface

The prototype uses an ILI9341 2.8-inch 320×240 pixel SPI TFT display with an XPT2046 touch controller. Both devices share the SPI bus (MOSI: GPIO 23, MISO: GPIO 19, SCLK: GPIO 18), with the display chip select on GPIO 5 and the touch chip select on GPIO 21. Both are managed with the `TFT_eSPI` library, configured at 40 MHz for the display and 2.5 MHz for the touch digitiser. The display operates in landscape orientation (rotation 1), yielding a usable canvas of 320×240 pixels.

The device has no physical buttons; the touchscreen is the sole user interface. `Display.ino` initialises the hardware and applies the touch calibration coefficients at boot via `tft.setTouch(calData)`, where `calData` holds five values in the format `{xMin, xMax, yMin, yMax, swapXY}`. Without a prior `setTouch()` call, `tft.getTouch()` returns zero coordinates; raw ADC values remain accessible through `getTouchRaw()` for hardware diagnostics.

`TouchUI.ino` implements a four-state machine updated on every loop iteration.

Screensaver (`UI_SCREEN_SCREENSAVER`) The system enters the screensaver at boot and after 30 seconds of inactivity (`UI_IDLE_TIMEOUT_MS = 30000 ms`) in any other state. The screensaver renders an animated cat face using `TFT_eSPI` primitives on a black background, with the eyes blinking every 3 seconds (`CAT_BLINK_INTERVAL_MS = 3000 ms`). A “Toca per desbloquejar” prompt is shown at the bottom of the screen. Any touch event transitions to the unlock screen.

Unlock screen (`UI_SCREEN_UNLOCK`) Access to the main menu is gated by a four-corner sequential pattern to prevent accidental dispensing. The screen displays four 70×70 pixel touch targets (`CORNER_HIT_ZONE_PX = 70`) at the corners of the panel, labelled TL, TR, BL, and BR. The user

must tap them in strict order: top-left → top-right → bottom-left → bottom-right. A numeric progress counter ($N / 4$) is rendered at the centre, and each correctly tapped corner highlights in green as visual feedback. If the sequence is not completed within 5 seconds of the first tap (`CORNER_TIMEOUT_MS = 5000 ms`), the progress resets and the user must start again. If 30 seconds elapse without completing the pattern, the system returns to the screensaver.

Main menu (UI_SCREEN_MAIN) The main menu is divided horizontally into two touch zones at $x = 160$ pixels. Tapping the left half ($tx < 160$) activates MANUAL mode; tapping the right half ($tx \geq 160$) activates AUTO mode. The header bar displays the system name, current operating mode, and RTC time refreshed every second. The footer shows current tray weight, ambient temperature, and Wi-Fi connection status. After 30 seconds of no touch activity, the system returns to the screensaver.

Manual feed screen (UI_SCREEN_MANUAL_FEED) When MANUAL mode is selected, this screen displays the live load-cell reading updated every 250 ms, enabling the operator to monitor dispensed weight in real time. A back button returns to the main menu.

Touch calibration coefficients are stored in a `uint16_t calData[5]` array and applied once during `touchUIInit()`.

3.6.6 Cloud Integration and Companion Software

The ESP32 communicates with Supabase through Wi-Fi using the REST API over HTTP/HTTPS. The firmware does not use the Supabase real-time WebSocket channel. Instead, `Network.ino` performs periodic polling to keep the embedded behaviour deterministic and to avoid the overhead of maintaining a persistent TCP connection.

Table 16: Supabase tables used by the firmware.

Table	Direction	Purpose
<code>commands</code>	Read	Pending remote instructions such as dispense, tare, and reboot.
<code>schedules</code>	Read	Feeding schedules with time, target weight, and enabled flag, cached locally in RAM.
<code>feed_events</code>	Write	Log of every completed or failed dispense cycle.
<code>realtime_weight</code>	Write	Current scale reading, updated during standby.
<code>device_config</code>	Read	Per-device configuration that can override <code>config.h</code> defaults at runtime.
<code>devices</code>	Write	Heartbeat information, last-seen timestamp, and firmware version.
<code>notifications</code>	Write	Error events such as motor timeout, blockage, and sensor fault.

All HTTP requests include an authorisation bearer token and a JSON content-type header. The

API URL and anonymous key are stored as compile-time constants in `config.h`. The current firmware revision does not use EEPROM or NVS storage; configuration is refreshed from Supabase on every boot.

The web dashboard is a React application built with Vite and connected to the same Supabase project through the Supabase JavaScript client. It provides live weight, temperature, humidity, device status, a feed-now action, cat profile management, schedule editing, feed history, device information, calibration parameters, and timezone settings. Representative front-end and back-end captures are included in Appendix B.

VetGat is the AI assistant integrated into the dashboard. It runs through an n8n workflow that receives user messages by a POST webhook, retrieves context from Supabase, and sends the assembled context to the OpenAI API. The workflow can query the active cat profile, enabled schedules, recent feed events, seven-day intake summary, device status, and latest hopper weight. The assistant can answer questions about feeding history, remaining food, schedule adherence, and general feline nutrition, but it does not issue commands to the device.

3.6.7 Safety and Error Handling

The firmware includes four safety mechanisms. First, the motor timeout stops dispensing if `STATE_DISPENSING` lasts longer than 30 seconds without reaching the target mass. Second, the load-cell sanity check in `Scale.ino` verifies that the raw HX711 value is not zero and is not the constant 8388607 value associated with a floating DOUT line. If this check fails, dispensing is cancelled and a `sensor_fault` notification is posted to Supabase. Third, the DS3231 I2C read is protected by a 100 ms timeout. After three consecutive I2C failures, the system falls back to a software clock based on `millis()` and logs an `rtc_fault` event. Fourth, the DRV8825 `ENABLE` pin is driven HIGH whenever the system is idle or in error state, preventing unnecessary holding current and reducing driver heating during continuous operation.

3.6.8 Libraries Used

The main software dependencies are shown in Table 17. They were selected because they provide direct support for the selected display, motor driver, load-cell amplifier, real-time clock, environmental sensor, JSON handling, and network communication.

Table 17: Libraries used by the control software.

Library	Version tested	Purpose
TFT_eSPI (Bodmer)	2.5.x	ILI9488 SPI display and XPT2046 touch controller.
AccelStepper	1.64	Non-blocking stepper motor control with acceleration profiles.
HX711 (bogde)	0.7.5	24-bit ADC communication for the load cell.
RTCLib (Adafruit)	2.1.x	DS3231 I2C real-time clock.
DHT sensor library (Adafruit)	1.4.x	DHT22 temperature and humidity sensor.
ArduinoJson	6.x	JSON serialisation and deserialisation for Supabase REST requests.
HTTPCClient	ESP32 core	HTTP/HTTPS GET and POST requests.
WiFi	ESP32 core	Wi-Fi station mode management.

3.7 Functional Testing of the Prototype

The validation of the prototype is planned through a set of functional tests designed to verify the correct operation of the main subsystems. These tests are necessary to evaluate whether the feeder satisfies the project requirements and to identify possible limitations of the design.

The first test consists of verifying the dispensing mechanism. The objective is to confirm that the auger screw can move dry food from the container to the bowl without blocking or producing irregular flow. Several dispensing cycles should be performed with different quantities of food inside the container.

The second test evaluates portion accuracy. In this test, the feeder is programmed to dispense a predefined amount of food, and the real dispensed mass is measured using the load cell. The measured value is compared with the target value in order to calculate the dispensing error.

The third test checks the repeatability of the system. The same portion is dispensed several times under the same conditions, and the variation between portions is analysed. This test is important to determine whether the mechanism can provide consistent results.

The fourth test validates the scheduled feeding function. The RTC module and the ESP32 software are tested by programming different feeding times and verifying that the feeder activates at the correct moments.

The fifth test evaluates power consumption. The current consumption of the 12 V and 5 V lines should be measured during dispensing and standby operation. This allows the real energy consumption to be compared with the estimated values calculated previously.

The main validation tests are summarized in Table 18.

These tests will make it possible to evaluate the performance, reliability, and limitations of the prototype. If significant deviations are detected, the results will be used to improve the mechanical design, motor control, calibration process, or power management strategy.

Table 18: Planned functional tests for the automatic cat feeder

Test	Objective	Expected result
Dispensing mechanism test	Verify food movement through the auger screw	Food is dispensed without blockage
Portion accuracy test	Compare target mass with real mass	Low dispensing error
Repeatability test	Analyse variation between repeated portions	Similar mass in repeated cycles
Scheduled feeding test	Verify RTC-based activation	Feeder activates at programmed times
Power consumption test	Measure 12 V and 5 V consumption	Consumption within estimated range

4 Cost of the Project

This section presents the estimated cost of the automatic cat feeder project. The total cost has been divided into two main categories: material cost and engineering labour cost. The material cost includes all the components that were finally used to build the prototype, while the labour cost estimates the engineering hours dedicated to the design, development, manufacturing, programming, testing, and project management tasks.

The project was developed over 8 sprints, each one lasting two weeks. The team was composed of 4 members, and each member dedicated approximately 15 hours per sprint. Therefore, the total number of working hours was calculated as follows:

$$H_{total} = 8 \cdot 4 \cdot 15 = 480 \text{ h} \quad (22)$$

As a result, the estimated dedication per team member was:

$$H_{member} = \frac{480}{4} = 120 \text{ h/member} \quad (23)$$

The distribution of working hours by project area is shown in Table 19.

The estimated dedication per team member is shown in Table 20.

The estimated material cost of the prototype is shown in Table 21.

The material cost of the prototype is 159.94 €. This value only includes the components that were finally used in the development of the automatic cat feeder prototype. Components that were initially considered but not implemented in the final design were excluded from the final budget.

To estimate the labour cost, a standard engineering hourly rate of 25 €/h was considered. There-

Table 19: Estimated man-hours dedicated to the project.

Project area	Estimated hours	Percentage
Mechanical design and 3D printing	150 h	31.25%
Electrical and electronic design	110 h	22.92%
Programming and control	140 h	29.17%
Testing and validation	50 h	10.42%
Project management and documentation	30 h	6.25%
Total	480 h	100%

Table 20: Estimated man-hours per team member.

Team member	Estimated hours
Member 1	120 h
Member 2	120 h
Member 3	120 h
Member 4	120 h
Total	480 h

fore, the labour cost was calculated as:

$$C_{labour} = H_{total} \cdot C_{hour} \quad (24)$$

$$C_{labour} = 480 \cdot 25 = 12000 \text{ €} \quad (25)$$

The total cost of the project is obtained by adding the material cost and the labour cost:

$$C_{total} = C_{materials} + C_{labour} \quad (26)$$

$$C_{total} = 159.94 + 12000 = 12159.94 \text{ €} \quad (27)$$

The complete project budget is summarized in Table 22.

The material cost of the prototype is relatively low, with a total of 159.94 €. However, when the engineering work is included, the total project cost increases significantly to 12159.94 €. This is because most of the value of the project is related to the design process, mechanical development, electronics integration, programming, testing, and documentation.

In conclusion, the total estimated cost of the automatic cat feeder project is 12159.94 €. The prototype itself is inexpensive in terms of materials, but the complete engineering cost reflects the

Table 21: Material cost of the automatic cat feeder prototype.

Item	Description of the material	Qty.	Unit price [€]	Cost [€]	Provider
1	NEMA 17 stepper motor	1	14.00	14.00	Amazon
2	ESP32 WROOM-32	1	13.00	13.00	Amazon
3	Load Cell 1 kg + HX711	1	20.00	20.00	Amazon
4	DHT22 temperature and humidity sensor	1	9.99	9.99	Robotshop
5	PLA 1 kg	1	13.00	13.00	Amazon
6	Universal double-sided PCB, 9 x 15 cm, 10 units	1	10.17	10.17	Amazon
7	DRV8825 stepper motor driver	1	19.24	19.24	Robotshop
8	RTC DS3231	1	4.50	4.50	Amazon
9	SAI Mini UPS 6000 mAh	1	39.90	39.90	Amazon
10	3.5-inch LCD TFT touch display	1	16.14	16.14	Amazon
Total material cost				159.94	

Table 22: Total project budget.

Cost category	Cost [€]
Material cost	159.94
Labour cost	12000.00
Total project cost	12159.94

real effort required to design, manufacture, program, test, and document a functional mechatronic system.

5 Conclusions

The automatic cat feeder project has been a complete and valuable learning experience. During its development, different areas of mechatronics engineering were applied, including mechanical design, 3D printing, electronics, control systems, and programming. For this reason, the project was considered a suitable choice, as it allowed the integration of several technical disciplines into a single functional prototype.

One of the main conclusions is that the selected solution, based on an auger screw mechanism driven by a stepper motor, is appropriate for controlling the dispensing of dry cat food. In addition, the use of a load cell improves portion control by measuring the real amount of food dispensed. This makes the system more reliable than simpler solutions based only on gravity or dispensing time.

The project also made it possible to understand the importance of mechanical design in the correct operation of the system. The geometry of the food container, the auger screw, and the outlet area directly affects the food flow and the risk of blockage. Therefore, the prototype required an iterative design process, especially for the parts manufactured using 3D printing.

From the electrical and programming point of view, the project helped to consolidate knowledge about sensors, actuators, microcontrollers, motor drivers, and power management. The ESP32 was a suitable control unit for the prototype because it offers enough processing capacity, connectivity, and compatibility with the selected electronic components.

The main limitations of the prototype are related to the mechanical behaviour of the dry food, the calibration of the load cell, and the long-term reliability of the dispensing mechanism. Different kibble sizes and shapes can affect the food flow, and additional testing would be required to validate the system under prolonged use.

Overall, the project achieved its main objective: to design and develop a functional automatic cat feeder prototype. Future improvements could include a better enclosure design, mobile application integration, more advanced calibration, food level detection, and long-term reliability testing. The final result demonstrates that the proposed system is technically feasible and coherent with the initial objectives.

6 Bibliography

The following bibliography uses the same numerical labels that appear throughout the document.

- [1] Islam, M. S., Rahman, M. M., & Hasan, M. (2019). Design and implementation of an automatic pet feeder using microcontroller and real-time clock. *International Journal of Engineering Research and Technology*.
- [2] Suryadevara, N. K., Mukhopadhyay, S. C., Wang, R., & Rayudu, R. K. (2013). Forecasting the behavior of an elderly using wireless sensors data in a smart home. *Engineering Applications of Artificial Intelligence*, 26(10), 2641–2652.
- [3] Kurniawan, A., & Hidayat, R. (2018). Design of automatic pet feeder based on Arduino microcontroller. *Proceedings of Applied Engineering and Technology*.
- [4] Espressif Systems. (2024). *ESP8266EX Datasheet*. Espressif Systems.
- [5] Espressif Systems. (2024). *ESP32-WROOM-32 Datasheet*. Espressif Systems.
- [6] Phidgets Inc. (2023). *Load Cell Primer and Calibration Guide*. Phidgets Documentation.
- [7] Avia Semiconductor. (2018). *HX711: 24-Bit Analog-to-Digital Converter for Weigh Scales*. Avia Semiconductor Datasheet.
- [8] Pet Feeders Comparative Review. (2024). Comparative analysis of commercial smart pet feeders and proprietary app ecosystems. *Product Benchmarking Notes*.
- [10] Wernimont, S. M., Thompson, R. J., & Radosevich, J. (2020). Use of activity and feeding behaviour monitoring in companion-animal health assessment. *Journal of Feline Medicine and Surgery*.
- [12] PETLIBRO. (2024). *Granary Automatic Pet Feeder User Manual and Product Specifications*. PETLIBRO Documentation.
- [15] Sure Petcare. (2024). *SureFeed Microchip Pet Feeder Connect: Product Manual*. Sure Petcare Documentation.
- [16] Bogde, A. (2024). *HX711 Arduino Library Documentation*. Open-source library documentation.
- [17] Smith, S. W. (1999). *The Scientist and Engineer’s Guide to Digital Signal Processing*. California Technical Publishing.
- [23] Welch, G., & Bishop, G. (2006). *An Introduction to the Kalman Filter*. University of North Carolina at Chapel Hill.
- [24] StepperOnline. (2024). *NEMA 17 Stepper Motor Specifications and Torque Curves*. Stepper-Online Technical Documentation.
- [27] Texas Instruments. (2014). *DRV8825 Stepper Motor Controller IC Datasheet*. Texas Instruments.

- [28] McCauley, M. (2023). *AccelStepper Library Documentation*. AirSpayce.
- [30] Arduino. (2024). *Blink Without Delay and millis()-Based Timing*. Arduino Documentation.
- [31] Espressif Systems. (2024). *ESP-IDF FreeRTOS SMP and Task Management Guide*. Espressif Documentation.
- [32] FreeRTOS. (2024). *FreeRTOS Kernel Developer Documentation*. Amazon Web Services.
- [36] Barr, M., & Massa, A. (2006). *Programming Embedded Systems*. O'Reilly Media.
- [39] Supabase. (2024). *Supabase Product Documentation*. Supabase Documentation.
- [40] Benoit Blanchon. (2024). *ArduinoJson: Efficient JSON Serialization for Embedded Systems*. ArduinoJson Documentation.
- [43] Bodmer. (2024). *TFT_eSPI Library Documentation*. Open-source library documentation.
- [44] Paul Stoffregen. (2024). *XPT2046-Touchscreen Library Documentation*. Open-source library documentation.
- [49] Ligout, S., Si, X., Vlaeminck, H., & Lyn, S. (2020). Cats reorganise their feeding behaviours when moving from ad libitum to restricted feeding. *Journal of Feline Medicine and Surgery*.
- [50] World Small Animal Veterinary Association. (2011). *WSAVA Global Nutrition Guidelines*. WSAVA Global Nutrition Committee.
- [52] American Animal Hospital Association & American Association of Feline Practitioners. (2021). *AAHA/AAFP Feline Life Stage Guidelines*. AAHA and AAFP.
- [53] National Research Council. (2006). *Nutrient Requirements of Dogs and Cats*. National Academies Press.
- [54] World Small Animal Veterinary Association. (2021). *Calorie Needs for Healthy Adult Cats*. WSAVA Global Nutrition Toolkit.
- [55] Pet Nutrition Alliance. (2024). *Calorie Calculator for Cats and Dogs*. Pet Nutrition Alliance.
- [56] German, A. J. (2006). The growing problem of obesity in dogs and cats. *Journal of Nutrition*, 136(7), 1940S–1946S.
- [57] Laflamme, D. P. (2006). Understanding and managing obesity in dogs and cats. *Veterinary Clinics of North America: Small Animal Practice*, 36(6), 1283–1295.
- [58] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- [60] PostgreSQL Global Development Group. (2024). *PostgreSQL Documentation*. PostgreSQL Documentation.
- [61] PostgREST. (2024). *PostgREST Documentation*. PostgREST Project.
- [62] Supabase. (2024). *Supabase REST API Documentation*. Supabase Documentation.
- [63] Supabase. (2024). *Row Level Security Guide*. Supabase Documentation.

- [64] Supabase. (2024). *Realtime Documentation*. Supabase Documentation.
- [69] n8n GmbH. (2024). *n8n Workflow Automation Documentation*. n8n Documentation.
- [71] Supabase. (2024). *pgvector and Vector Embeddings Guide*. Supabase Documentation.
- [72] OpenAI. (2024). *Embeddings and Retrieval-Augmented Generation Developer Guides*. OpenAI Documentation.
- [73] OpenAI. (2026). *ChatGPT* [Large language model]. OpenAI. <https://chat.openai.com/>

Declaration of AI Use

During the development of this project, artificial intelligence tools were used as support tools for brainstorming, text drafting, language correction, document structuring, and improvement of technical explanations. The final content, design decisions, calculations, prototype development, component selection, testing process, and validation of the project were reviewed, adapted, and approved by the project team. The AI tool was used as an auxiliary writing and support resource, but the responsibility for the final document and project outcomes belongs entirely to the authors.

A Project Code Annexes

This appendix contains the main software annexes for the embedded firmware, web application, cloud backend integration, and AI workflow of the CatFeeder-T5 system.

Project Code Annexes

This annex contains the core software implementation for the CatFeeder-T5 system. The source code is organized as follows:

1. Cloud Backend & AI Integration

The system uses Supabase as a real-time database and authentication provider. High-level cognitive features are implemented using n8n AI Agent workflow.

1.1. Supabase Client Configuration ('supabase.js')

This file configures the connection client to the Supabase backend. It enables automatic token refresh and session persistence.

```
```javascript
import { createClient } from '@supabase/supabase-js'

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY

export const supabase = createClient(supabaseUrl, supabaseAnonKey, {
 auth: {
 persistSession: true,
 autoRefreshToken: true,
 },
})
```
```

1.2. n8n AI Agent Workflow ('My workflow 4.json')

The n8n workflow exposes a webhook URL called by the front-end chat page. It routes user prompts through the AI agent workflow.

```
```json
{
 "name": "My workflow 4",
 "nodes": [
 {
 "parameters": {
 "httpMethod": "POST",
 "path": "69ddccfe-f7be-44b9-adb9-7471dbdc24e4",
 "responseMode": "responseNode",
 "options": {}
 },
 "type": "n8n-nodes-base.webhook",
 "typeVersion": 2.1,
 "position": [
 -160,

```



```

 -16
],
 "id": "c960ad06-410f-4a6c-a824-0e0ff3ee4cff",
 "name": "Webhook",
 "webhookId": "69ddccfe-f7be-44b9-adb9-7471dbdc24e4"
},
{
 "parameters": {
 "promptType": "define",
 "text": "={{ $json.body.chatInput }}",
 "options": {
 "systemMessage": "=Ets VetGat, un assistent especialitzat en salut i nutrició felina connecta"
 }
 },
 "type": "@n8n/n8n-nodes-langchain.agent",
 "typeVersion": 3.1,
 "position": [
 64,
 -16
],
 "id": "8ee19477-4953-43fd-bc31-51b6ec5adca9",
 "name": "AI Agent"
},
{
 "parameters": {
 "model": {
 "__rl": true,
 "value": "gpt-4o",
 "mode": "list",
 "cachedResultName": "gpt-4o"
 },
 "builtInTools": {},
 "options": {}
 },
 "type": "@n8n/n8n-nodes-langchain.lmChatOpenAi",
 "typeVersion": 1.3,
 "position": [
 -80,
 208
],
 "id": "510442d1-bb61-4098-8241-09edbbbc4ae48",
 "name": "OpenAI Chat Model",
 "credentials": {
 "openAiApi": {
 "id": "5g1S9zdyfwd8BRis",
 "name": "OpenAi account 2"
 }
 }
},
{
 "parameters": {
 "sessionIdType": "customKey",
 "sessionId": "={{ $json.body.sessionId }}",
 "contextWindowLength": 8
 }
}

```

```

 },
 "type": "@n8n/n8n-nodes-langchain.memoryBufferWindow",
 "typeVersion": 1.3,
 "position": [
 80,
 208
],
 "id": "b90f7ade-4add-4770-bee3-1d1a049c04f3",
 "name": "Simple Memory"
 },
 {
 "parameters": {
 "descriptionType": "manual",
 "toolDescription": "=Obté el perfil complet d'un gat de la base de dades. \nCrida aquesta tool",
 "operation": "get",
 "tableId": "cats",
 "filters": {
 "conditions": [
 {
 "keyName": "name",
 "keyValue": "={{ $fromAI('name') }}"
 },
 {
 "keyName": "archived",
 "keyValue": "=false"
 }
]
 }
 },
 "type": "n8n-nodes-base.supabaseTool",
 "typeVersion": 1,
 "position": [
 224,
 208
],
 "id": "59cc4091-f0ba-4352-b76e-c9f89b527ae8",
 "name": "get_cat_profile",
 "credentials": {
 "supabaseApi": {
 "id": "iy8oqaBXuNv3Bbji",
 "name": "Supabase account 2"
 }
 }
 },
 {
 "parameters": {
 "descriptionType": "manual",
 "toolDescription": "=Obté els horaris d'alimentació programats per a un gat concret.\nMostra a",
 "operation": "get",
 "tableId": "schedules",
 "filters": {
 "conditions": [
 {
 "keyName": "cat_id",

```

```

 "keyValue": "={{ $fromAI('cat_id') }}"
 },
 {
 "keyName": "enabled",
 "keyValue": "true"
 }
]
},
"type": "n8n-nodes-base.supabaseTool",
"typeVersion": 1,
"position": [
 368,
 208
],
"id": "c01c5791-4aa4-4433-afa5-fd4ba61dfd8e",
"name": "get_cat_schedules",
"credentials": {
 "supabaseApi": {
 "id": "iy8oqaBXuNv3Bbji",
 "name": "Supabase account 2"
 }
}
},
{
 "parameters": {
 "descriptionType": "manual",
 "toolDescription": "=Obté els darrers àpats registrats d'un gat: hora, grams programats \n(targ",
 "operation": "get",
 "tableId": "feed_events",
 "filters": {
 "conditions": [
 {
 "keyName": "cat_id",
 "keyValue": "={{ $fromAI('conditions0_Field_Value', ' ', 'string') }}"
 },
 {
 "keyName": "status",
 "keyValue": "completed"
 }
]
 }
 }
},
"type": "n8n-nodes-base.supabaseTool",
"typeVersion": 1,
"position": [
 544,
 208
],
"id": "0b582878-1d69-4b0e-9d70-0891eb5ff99f",
"name": "get_recent_feed_events",
"credentials": {
 "supabaseApi": {
 "id": "iy8oqaBXuNv3Bbji",

```

```

 "name": "Supabase account 2"
 }
}
},
{
 "parameters": {
 "descriptionType": "manual",
 "toolDescription": "=Calcula el total de grams consumits per un gat en els darrers dies, \nagrupa",
 "operation": "getAll",
 "tableId": "feed_events",
 "limit": "={{ /*n8n-auto-generated-fromAI-override*/ $fromAI('Limit', '', 'number') }}",
 "filters": {
 "conditions": [
 {
 "keyName": "cat_id",
 "condition": "eq",
 "keyValue": "={{ $fromAI('cat_id') }}"
 },
 {
 "keyName": "status",
 "condition": "eq",
 "keyValue": "completed"
 },
 {
 "keyName": "started_at",
 "condition": "gte",
 "keyValue": "={{ DateTime.now().minus({days:7}).toISO() }}"
 }
]
 }
 },
 "type": "n8n-nodes-base.supabaseTool",
 "typeVersion": 1,
 "position": [
 224,
 384
],
 "id": "93cb61e8-3b88-4686-a500-964b279153be",
 "name": "get_daily_intake_summary",
 "credentials": {
 "supabaseApi": {
 "id": "iy8oqaBXuNv3Bbji",
 "name": "Supabase account 2"
 }
 }
}
},
{
 "parameters": {
 "descriptionType": "manual",
 "toolDescription": "=Comprova l'estat del dispositiu d'alimentació associat a un gat.\nRetorna s",
 "operation": "getAll",
 "tableId": "devices",
 "limit": "={{ /*n8n-auto-generated-fromAI-override*/ $fromAI('Limit', '', 'number') }}",
 "filters": {

```

```

 "conditions": [
 {
 "keyName": "id",
 "condition": "eq",
 "keyValue": "=(SELECT device_id FROM schedules WHERE cat_id = '{{ $fromAI(\"cat_id\") }}'"
 }
]
 },
 "type": "n8n-nodes-base.supabaseTool",
 "typeVersion": 1,
 "position": [
 400,
 384
],
 "id": "4361945d-d538-4bfa-a848-7ce8ad6cd91a",
 "name": "get_device_status",
 "credentials": {
 "supabaseApi": {
 "id": "iy8oqaBXuNv3Bbji",
 "name": "Supabase account 2"
 }
 }
},
{
 "parameters": {
 "descriptionType": "manual",
 "toolDescription": "=Obté el pes actual del menjar dins el distribuïdor (en grams).\nUsa'l per s",
 "operation": "get",
 "tableId": "realtime_weight",
 "filters": {
 "conditions": [
 {
 "keyName": "device_id",
 "keyValue": "={{ $fromAI('device_id') }}"
 }
]
 }
 },
 "type": "n8n-nodes-base.supabaseTool",
 "typeVersion": 1,
 "position": [
 560,
 384
],
 "id": "e0bf7d78-2b51-49ad-82de-42a2d628ba41",
 "name": "get_hopper_weight",
 "credentials": {
 "supabaseApi": {
 "id": "iy8oqaBXuNv3Bbji",
 "name": "Supabase account 2"
 }
 }
},
},

```

```

{
 "parameters": {
 "options": {}
 },
 "type": "n8n-nodes-base.respondToWebhook",
 "typeVersion": 1.5,
 "position": [
 416,
 -16
],
 "id": "433e30f7-8c2e-48ef-b8e6-188a1a11223c",
 "name": "Respond to Webhook"
}
],
"pinData": {},
"connections": {
 "Webhook": {
 "main": [
 [
 {
 "node": "AI Agent",
 "type": "main",
 "index": 0
 }
]
]
 },
 "OpenAI Chat Model": {
 "ai_languageModel": [
 [
 {
 "node": "AI Agent",
 "type": "ai_languageModel",
 "index": 0
 }
]
]
 },
 "Simple Memory": {
 "ai_memory": [
 [
 {
 "node": "AI Agent",
 "type": "ai_memory",
 "index": 0
 }
]
]
 },
 "get_cat_profile": {
 "ai_tool": [
 [
 {
 "node": "AI Agent",

```

```

 "type": "ai_tool",
 "index": 0
 }
]
},
"get_cat_schedules": {
 "ai_tool": [
 [
 {
 "node": "AI Agent",
 "type": "ai_tool",
 "index": 0
 }
]
]
},
"get_recent_feed_events": {
 "ai_tool": [
 [
 {
 "node": "AI Agent",
 "type": "ai_tool",
 "index": 0
 }
]
]
},
"get_daily_intake_summary": {
 "ai_tool": [
 [
 {
 "node": "AI Agent",
 "type": "ai_tool",
 "index": 0
 }
]
]
},
"get_device_status": {
 "ai_tool": [
 [
 {
 "node": "AI Agent",
 "type": "ai_tool",
 "index": 0
 }
]
]
},
"get_hopper_weight": {
 "ai_tool": [
 [
 {

```

```

 "node": "AI Agent",
 "type": "ai_tool",
 "index": 0
 }
]
],
"AI Agent": {
 "main": [
 [
 {
 "node": "Respond to Webhook",
 "type": "main",
 "index": 0
 }
]
]
},
"Respond to Webhook": {
 "main": [
 []
]
}
},
"active": true,
"settings": {
 "executionOrder": "v1",
 "binaryMode": "separate",
 "availableInMCP": false
},
"versionId": "952d3416-e287-4741-9818-f4020702c041",
"meta": {
 "templateCredsSetupCompleted": true,
 "instanceId": "72ca13be64d15e8dd81551769c8c432039a52625acd9c2df019b7b7221f27d72"
},
"id": "0bZu5vRBINVgbBI",
"tags": []
}
'''

```

---

## ## 2. Frontend Web Application

The user-facing client is a single page application built on React, Vite, and Zustand for state management.

### ### 2.1. Global Zustand Store ('store.js')

Manages client-side cache and encapsulates Supabase database queries.

```

'''javascript
import { create } from 'zustand'
import { supabase } from '../lib/supabase'

```



```

export const useAuthStore = create((set) => ({
 user: null,
 session: null,
 loading: true,
 setUser: (user) => set({ user }),
 setSession: (session) => set({ session, user: session?.user ?? null }),
 setLoading: (loading) => set({ loading }),
 signOut: async () => {
 await supabase.auth.signOut()
 set({ user: null, session: null })
 }
}))

export const useAppStore = create((set, get) => ({
 cats: [],
 devices: [],
 schedules: [],
 notifications: [],
 toasts: [],
 activeFeed: null,

 setCats: (cats) => set({ cats }),
 setDevices: (devices) => set({ devices }),
 setSchedules: (schedules) => set({ schedules }),
 setNotifications: (notifications) => set({ notifications }),
 setActiveFeed: (activeFeed) => set({ activeFeed }),

 addToast: (message, type = 'info') => {
 const id = Date.now()
 set((s) => ({ toasts: [...s.toasts, { id, message, type }] }))
 setTimeout(() => {
 set((s) => ({ toasts: s.toasts.filter(t => t.id !== id) }))
 }, 3500)
 },
 removeToast: (id) => set((s) => ({ toasts: s.toasts.filter(t => t.id !== id) })),

 fetchCats: async () => {
 const { data } = await supabase.from('cats').select('*').eq('archived', false).order('created_at')
 if (data) set({ cats: data })
 },
 fetchDevices: async () => {
 const { data } = await supabase.from('devices').select('*').order('created_at')
 if (data) set({ devices: data })
 },
 fetchSchedules: async () => {
 const { data } = await supabase.from('schedules').select('*', cats(name)').order('time_of_day')
 if (data) set({ schedules: data })
 },
 fetchNotifications: async () => {
 const { data } = await supabase.from('notifications').select('*').eq('read', false).order('created_at')
 if (data) set({ notifications: data })
 },
}))
'''

```

### ### 2.2. Conversational AI Chat Component ('ChatPage.jsx')

Implements the user interface for sending messages to the n8n webhook and rendering responses from the V

```
``javascript
import { useState, useRef, useEffect } from 'react'
import { useAuthStore } from '../store/store'
import './ChatPage.css'

export default function ChatPage() {
 const { user } = useAuthStore()
 const [messages, setMessages] = useState([
 { id: 'initial', role: 'bot', text: 'Hi there! I am the CatFeeder AI assistant.\nAsk me anything about' }
])
 const [input, setInput] = useState('')
 const [isLoading, setIsLoading] = useState(false)
 const messagesEndRef = useRef(null)

 const scrollToBottom = () => {
 messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' })
 }

 useEffect(() => {
 scrollToBottom()
 }, [messages, isLoading])

 const handleSubmit = async (e) => {
 e.preventDefault()
 if (!input.trim() || isLoading) return

 const userMessage = {
 id: Date.now().toString(),
 role: 'user',
 text: input.trim()
 }

 setMessages(prev => [...prev, userMessage])
 setInput('')
 setIsLoading(true)

 try {
 const response = await fetch('https://micheline-aeonian-paula.ngrok-free.dev/webhook/69ddccfe-f7b
 method: 'POST',
 headers: {
 'Content-Type': 'application/json',
 'Accept': 'application/json'
 },
 body: JSON.stringify({
 sessionId: user?.id || 'anonymous',
 chatInput: userMessage.text
 })
 })
 }
 }
}
```

```

 if (!response.ok) throw new Error('Network response was not ok')

 const textResponse = await response.text();
 let botText = textResponse;

 try {
 const jsonData = JSON.parse(textResponse);
 if (Array.isArray(jsonData) && jsonData.length > 0) {
 botText = jsonData[0].output || jsonData[0].text || jsonData[0].message || JSON.stringify(j
 } else if (jsonData && typeof jsonData === 'object') {
 botText = jsonData.output || jsonData.text || jsonData.message || jsonData.response || JSON
 }
 } catch (e) {
 }

 setMessages(prev => [...prev, {
 id: Date.now().toString() + 'bot',
 role: 'bot',
 text: botText || 'Received empty response from the bot.'
 }])

 } catch (error) {
 console.error("Chat error:", error)
 setMessages(prev => [...prev, {
 id: Date.now().toString() + 'err',
 role: 'bot',
 text: 'Sorry, I failed to connect to the webhook. Please try again.'
 }])
 } finally {
 setIsLoading(false)
 }
}

return (
 <div className="page-enter chat-page">
 <div className="page-header" style={{ marginBottom: '16px' }}>
 <div>
 <h1>CatFeeder Agent</h1>
 <div className="subtitle">Chat with our AI to analyze and get insights about your cats.</div>
 </div>
 </div>

 <div className="chat-messages">
 {messages.map((msg) => (
 <div key={msg.id} className={`message ${msg.role}`}>
 <div className="message-bubble">
 {msg.text}
 </div>
 </div>
))}
 {isLoading && (
 <div className="message bot">
 <div className="message-bubble" style={{ padding: '16px' }}>
 <div className="typing-indicator">

```

```


 </div>
 </div>
 </div>
)}
<div ref={messagesEndRef} />
</div>

<form className="chat-input-form" onSubmit={handleSubmit}>
 <input
 type="text"
 className="form-input"
 placeholder="Ask something about your cats..."
 value={input}
 onChange={(e) => setInput(e.target.value)}
 disabled={isLoading}
 autoFocus
 />
 <button type="submit" className="btn btn-primary" disabled={isLoading || !input.trim()}>
 {isLoading ? '...' : 'Send'}
 </button>
</form>
</div>
)
}
'''

```

### ### 2.3. Dispenser Trigger Component ('FeedNowModal.jsx')

Provides the interface logic to write manual feeding commands to the Supabase database.

```

'''javascript
import { useState } from 'react'
import { supabase } from '../lib/supabase'

export default function FeedNowModal({ cats, device, onClose, onFed }) {
 const [catId, setCatId] = useState(cats[0]?.id ?? '')
 const [portion, setPortion] = useState(50)
 const [loading, setLoading] = useState(false)
 const [error, setError] = useState('')

 const handleFeed = async () => {
 if (!device) { setError('No device registered. Go to Device page first.');

```

```

 onFed(data)
 } catch (e) {
 setError(e.message)
 } finally {
 setLoading(false)
 }
}

return (
 <div className="modal-overlay" onClick={e => { if (e.target === e.currentTarget) onClose() }}>
 <div className="modal">
 <div className="modal-header">
 <h3>[bolt] Feed Now</h3>
 <button className="modal-close" onClick={onClose}>x</button>
 </div>
 <div className="modal-body">
 {error && <div className="auth-error">{error}</div>}
 <div className="form-group">
 <label className="form-label">Select Cat</label>
 <select className="form-select" value={catId} onChange={e => setCatId(e.target.value)}>
 {cats.map(c => <option key={c.id} value={c.id}>{c.name}</option>)}
 </select>
 </div>
 <div className="form-group">
 <label className="form-label">Portion Size: {portion}g</label>
 <input
 type="range"
 min={5} max={500} step={5}
 value={portion}
 onChange={e => setPortion(Number(e.target.value))}
 style={{width: '100%', accentColor: 'var(--accent)'}}
 />
 <div style={{display: 'flex', justifyContent: 'space-between', fontSize: '0.75rem', color: 'var(--text-muted)'}}>
 5g500g
 </div>
 </div>
 <div className="card" style={{padding: '14px 16px'}}>
 <div style={{fontSize: '0.82rem', color: 'var(--text-muted)', marginBottom: 4}}>Feed summary</div>
 <div style={{fontFamily: 'var(--font-display)', fontSize: '1.1rem'}}>
 {cats.find(c => c.id === catId)?.name ?? '---'} → <strong style={{color: 'var(--accent)'}}>{portion}g
 </div>
 </div>
 </div>
 <div className="modal-footer">
 <button className="btn btn-ghost" onClick={onClose}>Cancel</button>
 <button id="confirm-feed-btn" className="btn btn-primary" onClick={handleFeed} disabled={loading}>
 {loading ? '[dotted circle] Sending...' : '[bolt] Confirm Feed'}
 </button>
 </div>
 </div>
 </div>
)
}
'''

```

---

### ## 3. Embedded Firmware (ESP32 / Arduino)

The physical device uses an ESP32 micro-controller that links a load cell (HX711) and a stepper motor d

#### ### 3.1. Main System Header ('config.h')

Defines all global parameters, GPIO pins, data structures, and peripheral setups.

```
'''cpp
#ifndef CONFIG_H
#define CONFIG_H

#include <Arduino.h>
#include <TFT_eSPI.h>

extern TFT_eSPI tft;

#define FW_VERSION "0.2.0-touch"
#define SERIAL_BAUD 115200

#define WIFI_SSID "Lab-Modul"
#define WIFI_PASSWORD "GVe836Nf"
#define WIFI_RECONNECT_MS 10000
#define WIFI_CONNECT_TIMEOUT_MS 15000

#define SUPABASE_URL "https://jawqxuzlvvzsrobftupx.supabase.co"
#define SUPABASE_ANON_KEY \
 "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.\\" \
 "eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6Imphd3F4dXpsdnZ6c3JvYmZOdXB4Iiwicm9sZSI6Im\" \
 "Fub24iLCJpYXQiOiJlY2MjcsImV4cCI6MjA4OTQOMjYyN30.\" \
 "FmVhOjZ3Zj2XkIPlalwUbx3wAsaP4anr44Utrzs"
#define DEVICE_ID "44b0f051-549a-4b87-b67a-592254e5c84f"

#define DISPLAY_ROTATION 1
#define TOUCH_CS 21

#define I2C_SDA 32
#define I2C_SCL 33
#define I2C_CLOCK_HZ 400000

#define HX711_DT 16
#define HX711_SCK 17
#define HX711_GAIN 128
#define HX711_SAMPLES 5
#define HX711_CALIBRATION_FACTOR -2027.0f
#define SCALE_STABLE_THRESHOLD_G 0.3f

#define DHT_PIN 14
#define DHT_READ_INTERVAL_MS 2500

#define STEPPER_STEP 26
```

```

#define STEPPER_DIR 27
#define STEPPER_EN 25
#define STEPPER_STEPS_PER_REV 200
#define STEPPER_MICROSTEPS 16
#define STEPPER_MAX_SPEED 2000.0f
#define STEPPER_ACCEL 1500.0f
#define STEPPER_DISPENSE_SPEED 1200.0f
#define STEPPER_MAX_RUN_MS 30000
#define STEPPER_FEED_RATE_GPS 8.0f

#define DEFAULT_PORTION_G 25.0f
#define MIN_PORTION_G 5.0f
#define MAX_PORTION_G 500.0f
#define FEED_TOLERANCE_G 1.5f
#define FEED_STOP_THRESHOLD 0.90f
#define FEED_SETTLE_MS 500

#define TELEMETRY_INTERVAL_MS 2000
#define HEARTBEAT_INTERVAL_MS 30000
#define COMMAND_POLL_INTERVAL_MS 5000
#define CONFIG_POLL_INTERVAL_MS 60000
#define REALTIME_WEIGHT_INTERVAL_MS 500
#define DISPLAY_REFRESH_MS 250

enum DeviceState {
 STATE_BOOT,
 STATE_IDLE,
 STATE_DISPENSING,
 STATE_ERROR,
 STATE_OFFLINE_DEGRADED
};
extern DeviceState currentState;
extern const char *lastErrorMessage;

struct Telemetry {
 float weightG;
 float temperatureC;
 float humidity;
 bool wifiUp;
};
extern Telemetry telemetry;

struct FeedingCycle {
 bool active;
 float targetG;
 float dispensedG;
 float baselineG;
 uint32_t startMs;
 uint32_t lastPublishMs;
 const char *trigger;
 String commandId;
 String catId;
 String scheduleId;
 String startedAtIso;
};

```

```

};
extern FeedingCycle cycle;

enum UIMode {
 UI_MODE_MANUAL,
 UI_MODE_AUTO
};
extern UIMode uiRequestedMode;

enum UIScreen {
 UI_SCREEN_SCREENSAVER,
 UI_SCREEN_UNLOCK,
 UI_SCREEN_MAIN,
 UI_SCREEN_MANUAL_FEED,
 UI_SCREEN_AUTO_MODE
};
extern UIScreen activeUIScreen;
extern bool screenDirty;
extern uint8_t scheduleCount;

extern bool eyesOpen;
extern bool cornerHit[4];
extern uint8_t cornerHitCount;

extern uint32_t lastTouchMs;
extern uint32_t lastBlinkMs;
extern uint32_t lastPeriodicRedrawMs;
extern uint32_t lastManualDynamicMs;
extern uint32_t firstCornerMs;

#define UI_COL_BG 0x0820
#define UI_COL_CARD 0x1082
#define UI_COL_ACCENT 0x07FF
#define UI_COL_OK 0x07E0
#define UI_COL_DANGER 0xF800
#define UI_COL_WARN 0xFD20
#define UI_COL_TEXT 0xFFFF
#define UI_COL_MUTED 0x7BCF
#define UI_COL_OVERLAY 0x0000
#define UI_COL_CAT_LINE 0xC618

#define UI_IDLE_TIMEOUT_MS 30000
#define CAT_BLINK_INTERVAL_MS 3000
#define CORNER_HIT_ZONE_PX 56
#define CORNER_TIMEOUT_MS 5000
#define TOUCH_Z_THRESHOLD 350

#define MAIN_REDRAW_MS 1000
#define AUTO_REDRAW_MS 1000
#define MANUAL_DYNAMIC_MS 250

#define MANUAL_BTN_X 18
#define MANUAL_BTN_Y 146
#define MANUAL_BTN_W 284

```



```

#define MANUAL_BTN_H 74

#define MANUAL_BACK_W 96
#define HEADER_H 32
#define FOOTER_H 18

void touchUIInit();
void touchInputInit();
void touchInputUpdate();
void resetUnlockPattern();

void displayInit();
void displaySplash(const char *text);
void displayUpdate();
void transitionTo(UIScreen next);

void drawScreensaver();
void drawUnlockScreen();
void drawMainMenu();
void drawManualFeedScreen();
void drawManualFeedDynamic(bool force);
void drawAutoModeScreen();

void scaleInit();
void scaleTare();
float scaleRead();
void scaleCalibrate(float knownWeightG);

bool startDispense(float grams, const char *trigger, const String &commandId, const String &catId, const
void stopDispense();
void motorEmergencyStop();
void runDispensingCycle();
void checkScheduledFeeds();

struct DeviceTime {
 bool valid;
 uint8_t hour;
 uint8_t minute;
 uint8_t second;
 uint8_t wday;
 uint16_t yday;
};
extern DeviceTime currentDeviceTime;

#define TIMEZONE_OFFSET_SEC 7200
void updateTimeFromSupabase(const char* utcStr);
void updateLocalClock();

void sendHeartbeat();
void pollPendingCommands();
void pollDeviceConfig();
void publishRealtimeWeight(float weightG, float dispensedG, float targetG);
void logFeedEvent(const char *catId, float targetG, float actualG, const char *trigger, const char *sta
void updateCommandStatus(const String &cmdId, const char *status, float actualGrams);

```

```

#endif
'''

3.2. Shared Scheduling Data Structure ('schedules.h')

Defines the structure used to program feeding routines locally in the micro-controller's memory.

'''cpp
#ifndef SCHEDULES_H
#define SCHEDULES_H

#include <Arduino.h>

#define MAX_SCHEDULES 20

struct Schedule {
 bool enabled;
 uint8_t hour;
 uint8_t minute;
 uint8_t daysMask;
 float portionG;
 String catId;
 String id;
 uint16_t lastFiredYday;
};

#endif
'''

3.3. TFT Display SPI Pin Definition ('User_Setup.h')

Maps the ILI9341 display pins to the shared SPI bus lines of the ESP32.

'''cpp
#define USER_SETUP_INFO "ESP32_ILI9341_TOUCH"

#define ILI9341_DRIVER

#define TFT_WIDTH 240
#define TFT_HEIGHT 320

#define TFT_MOSI 23
#define TFT_MISO 19
#define TFT_SCLK 18

#define TFT_CS 5
#define TFT_DC 22
#define TFT_RST -1

#define TOUCH_CS 21

#define LOAD_GLCD
#define LOAD_FONT2

```

```

#define LOAD_FONT4

#define SPI_FREQUENCY 10000000
#define SPI_READ_FREQUENCY 6000000
#define SPI_TOUCH_FREQUENCY 1000000
'''

3.4. System Core Loop and Scheduler ('CatFeeder.ino')

Coordinated orchestrator that updates UI displays, checks schedules, executes non-blocking state loops,

'''cpp
#include "config.h"

#include <Arduino.h>
#include <SPI.h>
#include <WiFi.h>
#include <Wire.h>

#include <AccelStepper.h>
#include <ArduinoJson.h>
#include <DHT.h>
#include <HX711.h>
#include <TFT_eSPI.h>

#include "schedules.h"

TFT_eSPI tft = TFT_eSPI();
HX711 scale;
DHT dht(DHT_PIN, DHT22);
AccelStepper stepper(AccelStepper::DRIVER, STEPPER_STEP, STEPPER_DIR);

DeviceState currentState = STATE_BOOT;
const char *lastErrorMessage = "";

Telemetry telemetry = {0.0f, NAN, NAN, false};

FeedingCycle cycle = {false, 0.0f, 0.0f, 0.0f, 0, 0, "manual", "", "", "", ""};

extern uint8_t scheduleCount;

static uint32_t tLastSensors = 0;
static uint32_t tLastDisplay = 0;
static uint32_t tLastHeartbeat = 0;
static uint32_t tLastCmdPoll = 0;
static uint32_t tLastCfgPoll = 0;
static uint32_t tLastWifiTry = 0;
static uint32_t tLastWeightPublish = 0;
#define IDLE_WEIGHT_PUBLISH_MS 10000

void setup() {
 Serial.begin(SERIAL_BAUD);
 delay(150);
 Serial.println();

```

```

Serial.println(F("====="));
Serial.println(F(" CatFeeder --- firmware " FW_VERSION));
Serial.println(F("====="));

pinMode(STEPPER_EN, OUTPUT);
digitalWrite(STEPPER_EN, HIGH);

touchInputInit();

displayInit();
displaySplash("Booting...");

i2cInit();
scaleInit();
sensorsInit();
motorInit();
networkInit();

touchUIInit();

currentState = STATE_IDLE;
displaySplash("Ready");
delay(400);

Serial.println(F("[boot] complete"));
}

void loop() {
 const uint32_t now = millis();

 static uint32_t tLastTouchUpdate = 0;
 if (now - tLastTouchUpdate >= 60) {
 tLastTouchUpdate = now;
 touchInputUpdate();
 }

 switch (currentState) {
 case STATE_IDLE:
 if (uiRequestedMode == UI_MODE_AUTO) {
 checkScheduledFeeds();
 }
 break;
 case STATE_DISPENSING:
 runDispensingCycle();
 if (now - cycle.lastPublishMs >= REALTIME_WEIGHT_INTERVAL_MS) {
 cycle.lastPublishMs = now;
 if (telemetry.wifiUp) {
 publishRealtimeWeight(telemetry.weightG, cycle.dispensedG, cycle.targetG);
 }
 }
 break;
 case STATE_ERROR:
 break;
 default:

```

```

 break;
}

if (now - tLastSensors >= TELEMETRY_INTERVAL_MS) {
 tLastSensors = now;
 updateSensors();
}

if (now - tLastDisplay >= DISPLAY_REFRESH_MS) {
 tLastDisplay = now;
 displayUpdate();
}

if (WiFi.status() == WL_CONNECTED) {
 if (!telemetry.wifiUp) {
 telemetry.wifiUp = true;
 tLastHeartbeat = now;
 sendHeartbeat();
 tLastCfgPoll = now;
 pollDeviceConfig();
 tLastCmdPoll = now;
 pollPendingCommands();
 }
 if (now - tLastCmdPoll >= COMMAND_POLL_INTERVAL_MS) {
 tLastCmdPoll = now;
 pollPendingCommands();
 }
 if (now - tLastCfgPoll >= CONFIG_POLL_INTERVAL_MS) {
 tLastCfgPoll = now;
 pollDeviceConfig();
 }
 if (now - tLastHeartbeat >= HEARTBEAT_INTERVAL_MS) {
 tLastHeartbeat = now;
 sendHeartbeat();
 }
 if (!cycle.active && now - tLastWeightPublish >= IDLE_WEIGHT_PUBLISH_MS) {
 tLastWeightPublish = now;
 publishRealtimeWeight(telemetry.weightG, 0, 0);
 }
} else {
 telemetry.wifiUp = false;
 if (now - tLastWifiTry >= WIFI_RECONNECT_MS) {
 tLastWifiTry = now;
 networkReconnect();
 }
}
}

void i2cInit() {
 Wire.begin(I2C_SDA, I2C_SCL);
 Wire.setClock(I2C_CLOCK_HZ);
 Serial.println(F("[i2c] started"));
}

```

```

void goToError(const char *message) {
 lastErrorMessage = message;
 currentState = STATE_ERROR;
 motorEmergencyStop();
 Serial.print(F("[ERROR] "));
 Serial.println(message);
}

void clearError() {
 lastErrorMessage = "";
 currentState = STATE_IDLE;
 Serial.println(F("[ERROR] cleared"));
}

void cycleFinish(const char *status, float dispensed) {
 cycle.active = false;
 cycle.dispensedG = dispensed;
 motorEnable(false);

 if (telemetry.wifiUp) {
 logFeedEvent(cycle.catId.c_str(), cycle.targetG, dispensed,
 cycle.trigger, status);
 if (cycle.commandId.length() > 0) {
 updateCommandStatus(cycle.commandId, status, dispensed);
 cycle.commandId = "";
 }
 }

 currentState = STATE_IDLE;
 Serial.printf("[cycle] %s --- %.1fg dispensed\n", status, dispensed);
}
'''

```

### ### 3.5. REST Client Database Link ('Network.ino')

Establishes connection headers, executes API requests for telemetry publishing, claims incoming commands

```

'''cpp
#include <ArduinoJson.h>
#include <HttpClient.h>
#include <WiFiClientSecure.h>

static WiFiClientSecure netClient;

void loadSchedules(JsonArray &arr);
extern uint8_t scheduleCount;
void scaleTare();
float scaleRead();

static const char *HDR_APIKEY = "apikey";
static const char *HDR_AUTH = "Authorization";

void networkInit() {
 netClient.setInsecure();
}

```

```

WiFi.mode(WIFI_STA);
WiFi.setAutoReconnect(true);
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
Serial.printf("[wifi] connecting to %s ", WIFI_SSID);

uint32_t start = millis();
while (WiFi.status() != WL_CONNECTED &&
 millis() - start < WIFI_CONNECT_TIMEOUT_MS) {
 Serial.print('.');
 delay(250);
}
Serial.println();

if (WiFi.status() == WL_CONNECTED) {
 Serial.printf("[wifi] ok, IP=%s\n", WiFi.localIP().toString().c_str());
 telemetry.wifiUp = true;
} else {
 Serial.println(F("[wifi] failed --- continuing offline"));
 telemetry.wifiUp = false;
}
}

void networkReconnect() {
 if (WiFi.status() == WL_CONNECTED)
 return;
 Serial.println(F("[wifi] reconnecting..."));
 WiFi.disconnect();
 WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
}

static void applySupabaseHeaders(HTTPClient &http, bool patch = false) {
 http.addHeader(HDR_APIKEY, SUPABASE_ANON_KEY);
 http.addHeader(HDR_AUTH, String("Bearer ") + SUPABASE_ANON_KEY);
 http.addHeader("Content-Type", "application/json");
 if (patch)
 http.addHeader("Prefer", "return=minimal");
}

static int httpGET(const String &endpoint, String &out) {
 if (WiFi.status() != WL_CONNECTED)
 return -1;
 HTTPClient http;
 http.begin(netClient, String(SUPABASE_URL) + endpoint);
 applySupabaseHeaders(http);
 int code = http.GET();
 if (code > 0)
 out = http.getString();
 http.end();
 return code;
}

static int httpPOST(const String &endpoint, const String &body) {
 if (WiFi.status() != WL_CONNECTED)

```

```

 return -1;
 HTTPClient http;
 http.begin(netClient, String(SUPABASE_URL) + endpoint);
 applySupabaseHeaders(http);
 int code = http.POST(body);
 http.end();
 return code;
}

static int httpPATCH(const String &endpoint, const String &body) {
 if (WiFi.status() != WL_CONNECTED)
 return -1;
 HTTPClient http;
 http.begin(netClient, String(SUPABASE_URL) + endpoint);
 applySupabaseHeaders(http, true);
 int code = http.PATCH(body);
 http.end();
 return code;
}

static String isoTimestamp() {
 return String("now");
}

void sendHeartbeat() {
 JsonDocument doc;
 doc["last_seen"] = "now";
 doc["status"] = cycle.active
 ? "dispensing"
 : (currentState == STATE_ERROR ? "fault_motor" : "idle");
 doc["firmware_version"] = FW_VERSION;
 String body;
 serializeJson(doc, body);

 if (WiFi.status() != WL_CONNECTED)
 return;

 HTTPClient http;
 http.begin(netClient, String(SUPABASE_URL) + "/rest/v1/devices?id=req." DEVICE_ID);

 http.addHeader("apikey", SUPABASE_ANON_KEY);
 http.addHeader("Authorization", String("Bearer ") + SUPABASE_ANON_KEY);
 http.addHeader("Content-Type", "application/json");
 http.addHeader("Prefer", "return=representation");

 int code = http.PATCH(body);
 Serial.printf("[net] heartbeat HTTP %d\n", code);

 if (code == 200 || code == 201) {
 String payload = http.getString();
 JsonDocument respDoc;
 DeserializationError err = deserializeJson(respDoc, payload);
 if (!err && respDoc.is<JsonArray>() && respDoc.size() > 0) {
 const char* lastSeen = respDoc[0]["last_seen"] | "";

```



```

 if (lastSeen && lastSeen[0]) {
 updateTimeFromSupabase(lastSeen);
 }
 }
}
http.end();
}

void publishRealtimeWeight(float weightG, float dispensedG, float targetG) {
 if (WiFi.status() != WL_CONNECTED)
 return;
 JsonDocument doc;
 doc["device_id"] = DEVICE_ID;
 doc["weight_grams"] = weightG;
 String body;
 serializeJson(doc, body);

 HTTPClient http;
 http.begin(netClient, String(SUPABASE_URL) + "/rest/v1/realtime_weight");
 applySupabaseHeaders(http);
 http.addHeader("Prefer", "resolution=merge-duplicates");
 int code = http.POST(body);
 http.end();
 Serial.printf("[net] realtime_weight HTTP %d (%.1fg)\n", code, weightG);
}

void logFeedEvent(const char *catId, float targetG, float actualG,
 const char *trigger, const char *status) {
 JsonDocument doc;
 doc["device_id"] = DEVICE_ID;
 if (catId && catId[0])
 doc["cat_id"] = catId;
 if (cycle.scheduleId.length())
 doc["schedule_id"] = cycle.scheduleId;
 doc["target_grams"] = targetG;
 doc["actual_grams"] = actualG;
 doc["trigger_type"] = trigger;
 doc["status"] = status;
 doc["started_at"] = cycle.startedAtIso;

 String body;
 serializeJson(doc, body);
 int code = httpPOST("/rest/v1/feed_events", body);
 if (code < 200 || code >= 300) {
 Serial.printf("[net] feed_event HTTP %d\n", code);
 }
}

void updateCommandStatus(const String &cmdId, const char *status,
 float actualGrams) {
 JsonDocument doc;
 doc["status"] = status;
 String body;
 serializeJson(doc, body);

```

```

 String ep = "/rest/v1/commands?id=req." + cmdId;
 httpPATCH(ep, body);
}

void pollPendingCommands() {
 String ep = "/rest/v1/commands?device_id=req." DEVICE_ID
 "&status=req.pending&select=id,cat_id,portion_grams,command_type"
 "&limit=1";
 String payload;
 int code = httpGET(ep, payload);
 if (code != 200) {
 Serial.printf("[net] cmd poll HTTP %d\n", code);
 return;
 }
 if (payload.length() < 3) {
 Serial.println(F("[net] cmd poll: none pending"));
 return;
 }

 JsonDocument doc;
 DeserializationError err = deserializeJson(doc, payload);
 if (err || !doc.is<JsonArray>() || doc.size() == 0)
 return;

 if (currentState != STATE_IDLE)
 return;

 JsonObject cmd = doc[0];
 String id = cmd["id"] | "";
 String cat = cmd["cat_id"] | "";
 float grams = cmd["portion_grams"] | DEFAULT_PORTION_G;
 String type = cmd["command_type"] | "dispense";

 if (id.length() == 0)
 return;

 Serial.printf("[net] command %s, type=%s, %.1f g\n", id.c_str(), type.c_str(), grams);

 {
 JsonDocument up;
 up["status"] = "processing";
 String body;
 serializeJson(up, body);
 httpPATCH("/rest/v1/commands?id=req." + id, body);
 }

 if (type == "tare") {
 scaleTare();
 updateCommandStatus(id, "completed", 0.0f);
 telemetry.weightG = scaleRead();
 publishRealtimeWeight(telemetry.weightG, 0, 0);
 return;
 }
}

```

```

 if (!startDispense(grams, "manual", id, cat, String(""))) {
 updateCommandStatus(id, "error", 0.0f);
 }
}

void pollDeviceConfig() {
 String ep = "/rest/v1/schedules?enabled=eq.true"
 "&select=id,cat_id,time_of_day,days_of_week,portion_grams";
 String payload;
 int code = httpGET(ep, payload);
 if (code != 200 || payload.length() < 2)
 return;

 JsonDocument doc;
 if (deserializeJson(doc, payload))
 return;
 if (!doc.is<JsonArray>())
 return;

 JsonArray arr = doc.as<JsonArray>();
 loadSchedules(arr);
 Serial.printf("[net] schedules loaded: %u\n", scheduleCount);
}
'''

```

### ### 3.6. Non-Blocking Motor Control ('Motor.ino')

Implements step generation via hardware PWM (LEDC), applying a 90% gravimetric early cutoff threshold to

```

'''cpp
#define MOTOR_STEP_HZ 500
#define MOTOR_LEDC_BITS 8
#define MOTOR_DUTY_RUN 128
#define MOTOR_DUTY_STOP 0

#define DISPENSE_SCALE_MS 300UL
static uint32_t lastScaleCheckMs = 0;

static bool waitingFinalRead = false;
static uint32_t settleStartMs = 0;

void motorInit() {
 pinMode(STEPPER_EN, OUTPUT);
 pinMode(STEPPER_DIR, OUTPUT);
 digitalWrite(STEPPER_EN, HIGH);
 digitalWrite(STEPPER_DIR, LOW);

 ledcAttach(STEPPER_STEP, MOTOR_STEP_HZ, MOTOR_LEDC_BITS);
 ledcWrite(STEPPER_STEP, MOTOR_DUTY_STOP);

 Serial.println(F("[motor] stepper driver ready (LEDC hardware PWM)"));
}

void motorEnable(bool on) {

```

```

 digitalWrite(STEPPER_EN, on ? LOW : HIGH);
}

void motorEmergencyStop() {
 ledcWrite(STEPPER_STEP, MOTOR_DUTY_STOP);
 motorEnable(false);
 cycle.active = false;
 waitingFinalRead = false;
 Serial.println(F("[motor] emergency stop"));
}

void stopDispense() {
 if (currentState == STATE_DISPENSING && cycle.active && !waitingFinalRead) {
 ledcWrite(STEPPER_STEP, MOTOR_DUTY_STOP);
 motorEnable(false);
 waitingFinalRead = true;
 settleStartMs = millis();
 Serial.printf("[motor] manual dispense stopped --- settling %ums\n", (unsigned)FEED_SETTLE_MS);
 }
}

bool startDispense(float grams,
 const char *trigger,
 const String &commandId,
 const String &catId,
 const String &scheduleId) {
 if (currentState == STATE_DISPENSING || cycle.active) {
 Serial.println(F("[motor] rejected: already dispensing"));
 return false;
 }
 if (grams < MIN_PORTION_G || grams > MAX_PORTION_G) {
 Serial.printf("[motor] rejected: %.1f g out of range\n", grams);
 return false;
 }

 cycle.active = true;
 cycle.targetG = grams;
 cycle.dispensedG = 0.0f;
 cycle.baselineG = scaleRead();
 Serial.printf("[motor] baseline=%.1fg\n", cycle.baselineG);
 cycle.startMs = millis();
 cycle.lastPublishMs = 0;
 cycle.trigger = trigger;
 cycle.commandId = commandId;
 cycle.catId = catId;
 cycle.scheduleId = scheduleId;
 cycle.startedAtIso = isoTimestamp();

 waitingFinalRead = false;
 lastScaleCheckMs = millis();

 digitalWrite(STEPPER_DIR, LOW);
 motorEnable(true);
 delay(5);
}

```

```

 ledcWrite(STEP_PER_STEP, MOTOR_DUTY_RUN);

 currentState = STATE_DISPENSING;
 Serial.printf("[motor] dispensing %.1fg (%s) --- LEDC @%dHz, stop@%.0f%%\n",
 grams, trigger, MOTOR_STEP_HZ,
 FEED_STOP_THRESHOLD * 100.0f);
 return true;
}

void runDispensingCycle() {
 const uint32_t nowMs = millis();

 if (waitingFinalRead) {
 if (nowMs - settleStartMs >= FEED_SETTLE_MS) {
 float raw = cycle.baselineG + cycle.dispensedG;
 uint32_t tStart = millis();
 bool success = false;
 while (millis() - tStart < 150) {
 if (scale.is_ready()) {
 raw = scale.get_units(3);
 success = true;
 break;
 }
 delay(10);
 }
 float finalDispensed = raw - cycle.baselineG;
 if (finalDispensed < 0) finalDispensed = 0.0f;
 if (success) {
 telemetry.weightG = raw;
 }
 waitingFinalRead = false;
 cycleFinish("completed", finalDispensed);
 }
 return;
 }

 if (nowMs - lastScaleCheckMs >= DISPENSE_SCALE_MS && scale.is_ready()) {
 lastScaleCheckMs = nowMs;
 float raw = scale.get_units(1);
 float disp = raw - cycle.baselineG;
 if (disp < 0) disp = 0;
 cycle.dispensedG = disp;
 telemetry.weightG = raw;
 }

 const bool scaleReliable = (nowMs - cycle.startMs) >= 2000UL;

 if (scaleReliable && cycle.dispensedG >= cycle.targetG * FEED_STOP_THRESHOLD) {
 ledcWrite(STEP_PER_STEP, MOTOR_DUTY_STOP);
 motorEnable(false);
 waitingFinalRead = true;
 settleStartMs = nowMs;
 Serial.printf("[motor] reached %.0f%% (%.1f/%.1fg) --- settling %ums\n",

```

```

 FEED_STOP_THRESHOLD * 100.0f, cycle.dispensedG,
 cycle.targetG, (unsigned)FEED_SETTLE_MS);
 return;
}

if (nowMs - cycle.startMs > STEPPER_MAX_RUN_MS) {
 ledcWrite(STEPPER_STEP, MOTOR_DUTY_STOP);
 motorEmergencyStop();
 if (cycle.dispensedG > 0.5f) {
 cycleFinish("partial", cycle.dispensedG);
 } else {
 cycleFinish("error", 0.0f);
 goToError("Motor timeout");
 }
}
}
}
'''

3.7. Load Cell Weight Sensing ('Scale.ino')

Integrates with the HX711 24-bit ADC, averaging samples and filtering noise below a stable scale threshold

'''cpp
void scaleInit() {
 scale.begin(HX711_DT, HX711_SCK);

 uint32_t t0 = millis();
 while (!scale.is_ready() && millis() - t0 < 1500) {
 delay(50);
 }

 if (!scale.is_ready()) {
 Serial.println(F("[scale] HX711 not responding --- check wiring (continuing)"));
 return;
 }

 scale.set_scale(HX711_CALIBRATION_FACTOR);
 scale.tare(HX711_SAMPLES);
 Serial.printf("[scale] ready, factor=%.2f\n", HX711_CALIBRATION_FACTOR);
}

void scaleTare() {
 if (cycle.active) {
 Serial.println(F("[scale] tare rejected: cycle active"));
 return;
 }
 scale.tare(HX711_SAMPLES);
 Serial.println(F("[scale] tare done"));
}

float scaleRead() {
 if (!scale.is_ready()) return telemetry.weightG;
 float g = scale.get_units(HX711_SAMPLES);
 if (fabsf(g) < SCALE_STABLE_THRESHOLD_G) g = 0.0f;

```

```

 return g;
}

void scaleCalibrate(float knownWeightG) {
 Serial.println(F("[scale] calibrating..."));
 scale.set_scale();
 delay(500);
 long raw = scale.read_average(20);
 float newFactor = (float)raw / knownWeightG;
 scale.set_scale(newFactor);
 Serial.printf("[scale] new calibration factor = %.2f\n", newFactor);
 Serial.println(F("[scale] update HX711_CALIBRATION_FACTOR in config.h"));
}
'''

```

## B WebApp Front-End and Back-End ScreenShots

This appendix collects the screenshots about the web application front-end and back-end.

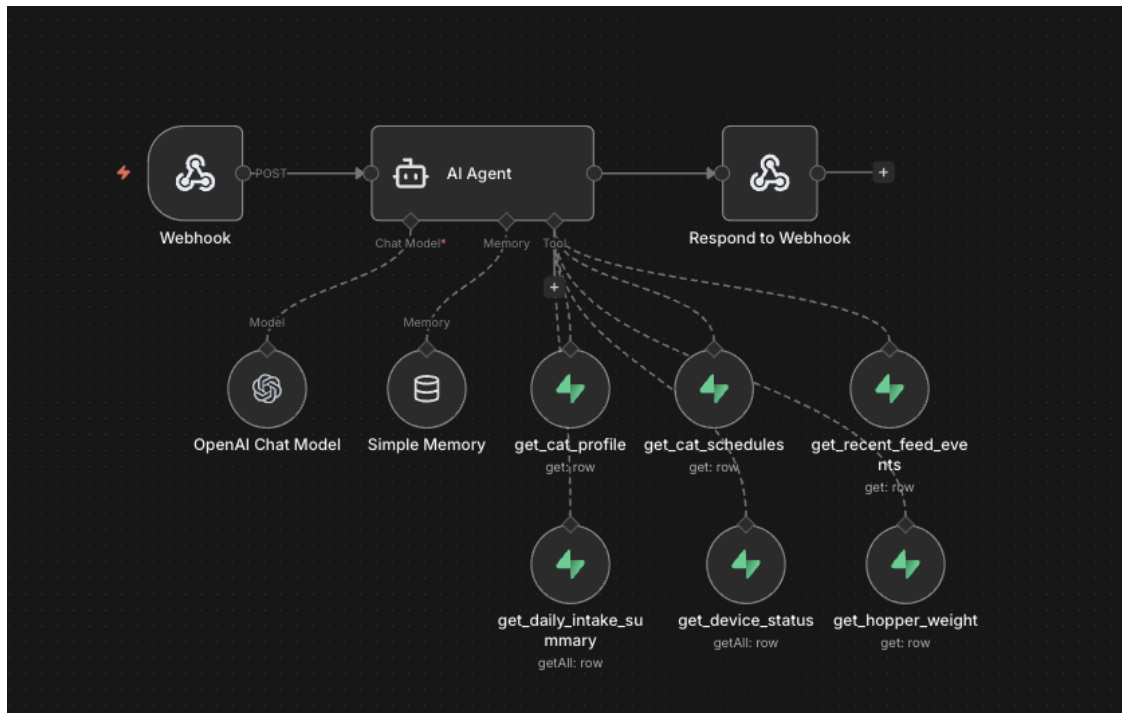


Figure 2: CatFeeder-T5 application capture 1.

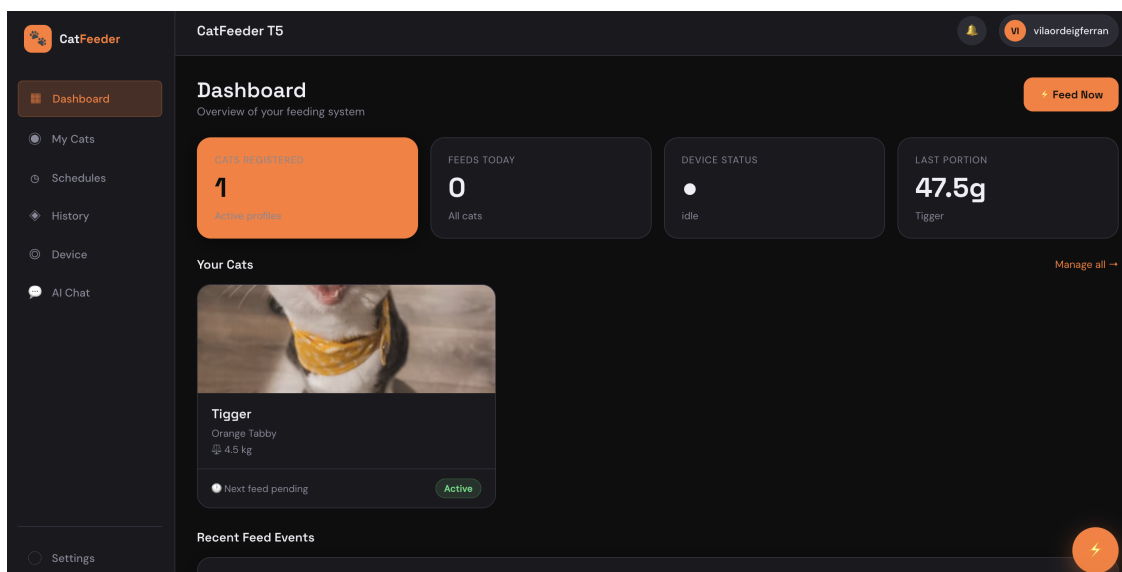


Figure 3: CatFeeder-T5 application capture 2.



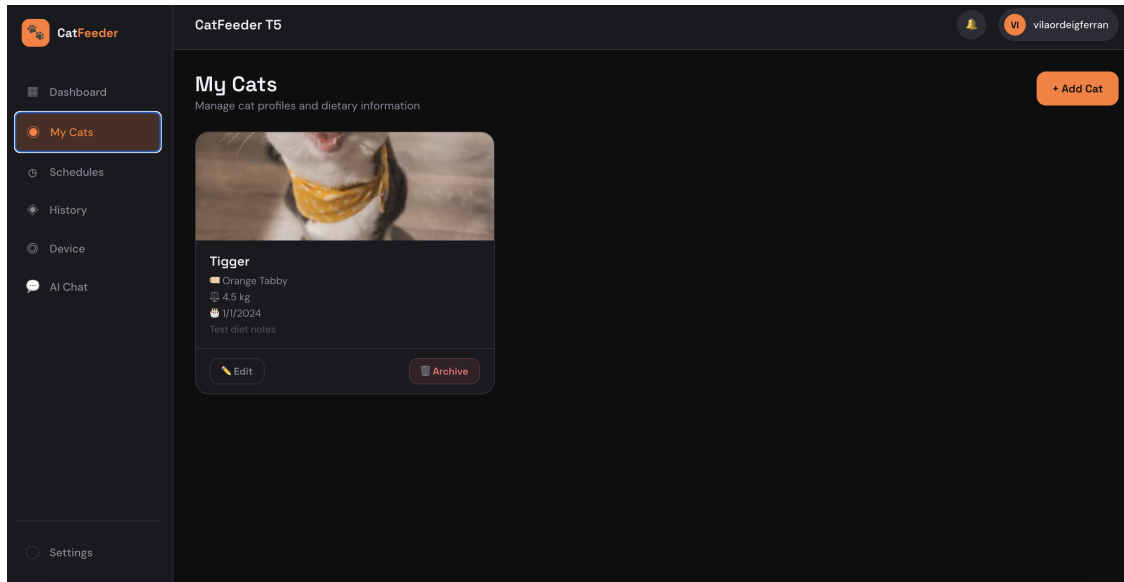


Figure 4: CatFeeder-T5 application capture 3.

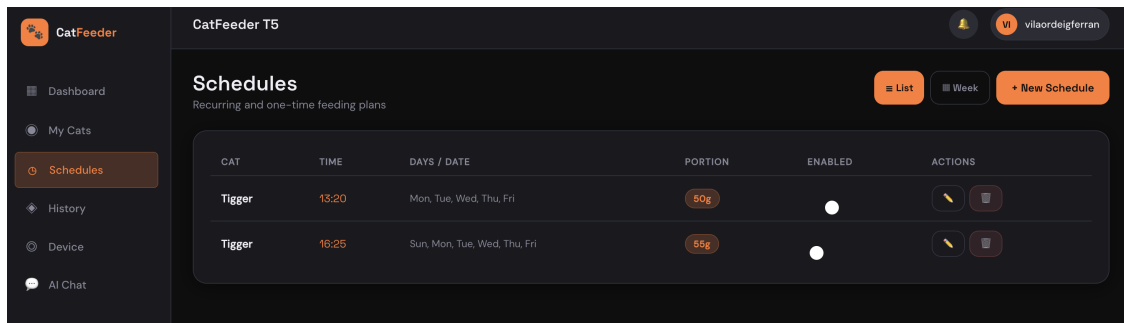


Figure 5: CatFeeder-T5 application capture 4.

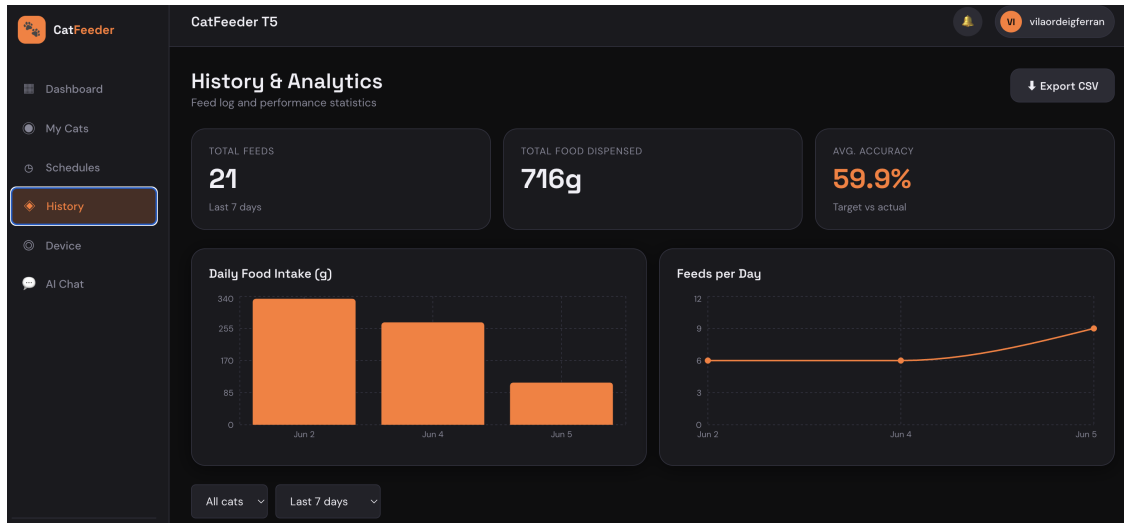


Figure 6: CatFeeder-T5 application capture 5.

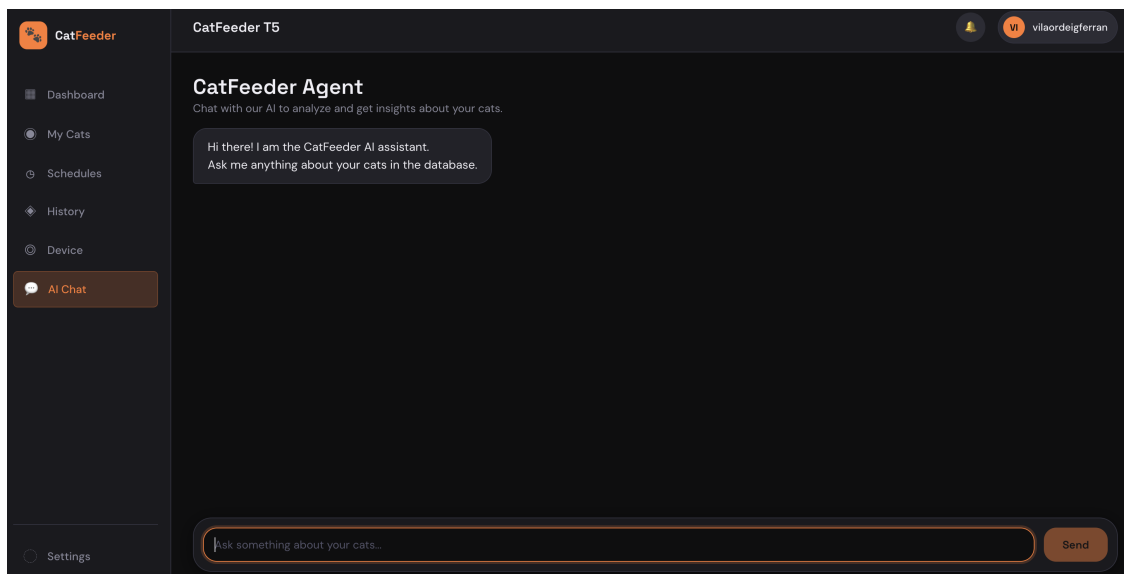


Figure 7: CatFeeder-T5 application capture 6.

## C Electrical Schematic

This appendix includes the complete electrical schematic of the automatic cat feeder prototype. The schematic shows the connections between the ESP32 WROOM-32 microcontroller, the DRV8825 stepper motor driver, the HX711 load cell amplifier, the DS3231 RTC module, the DHT22 sensor, the TFT touch display, and the power supply system.

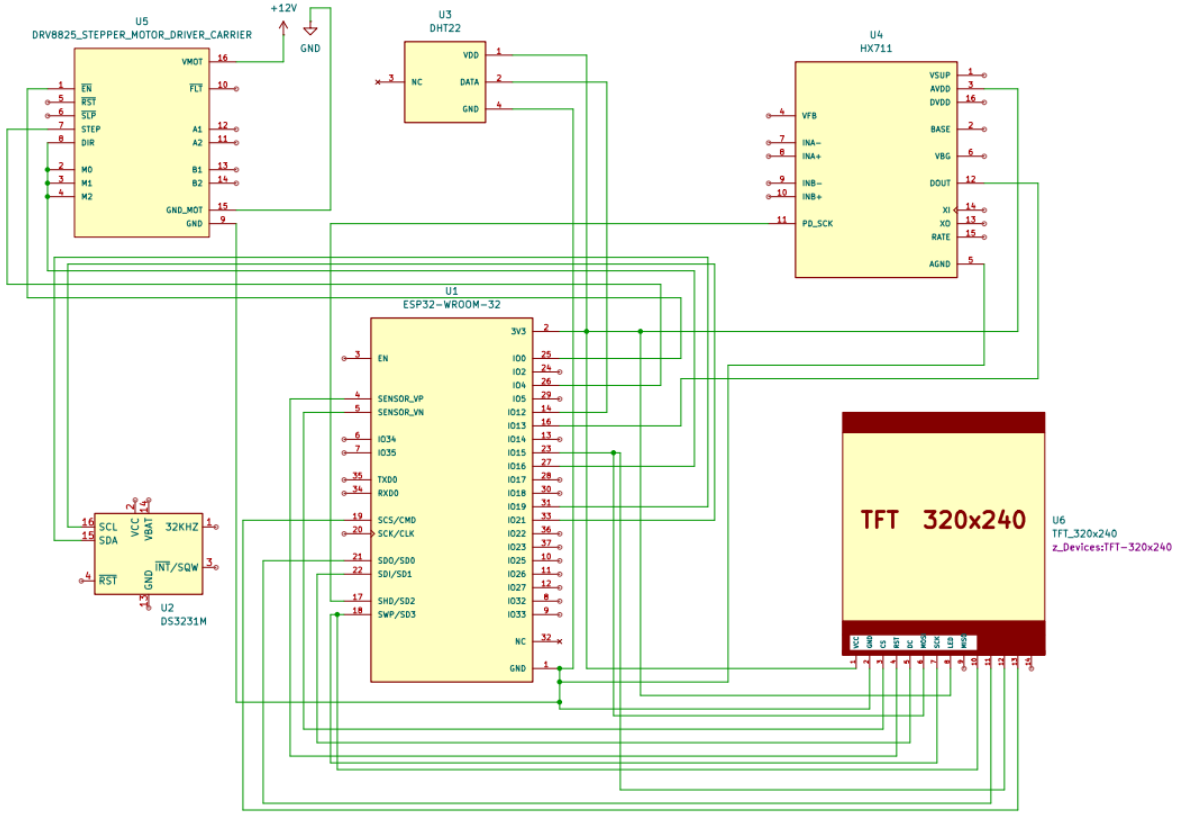


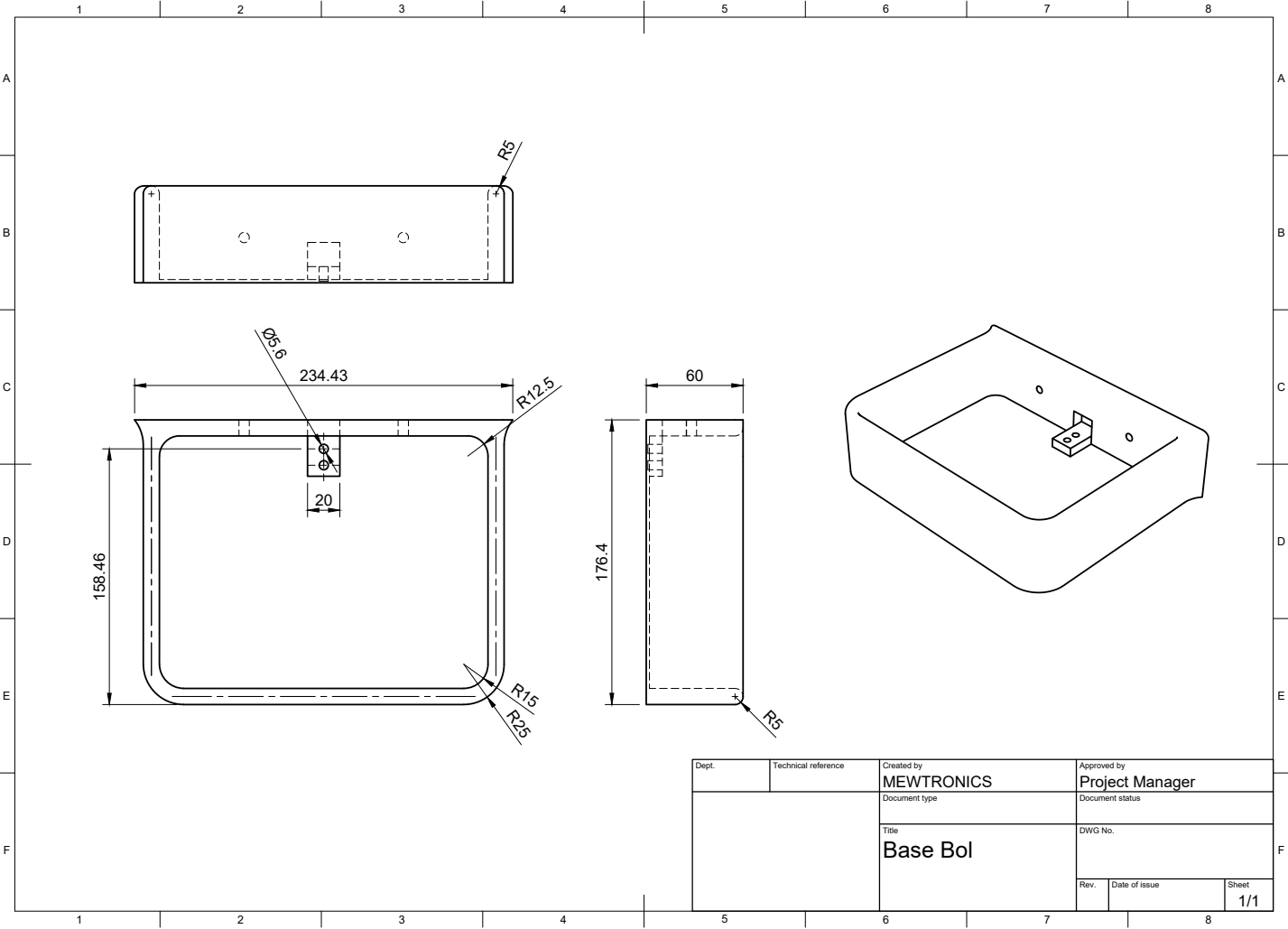
Figure 8: Electrical schematic of the automatic cat feeder prototype.

## D Dimensioned 3D Part Drawings

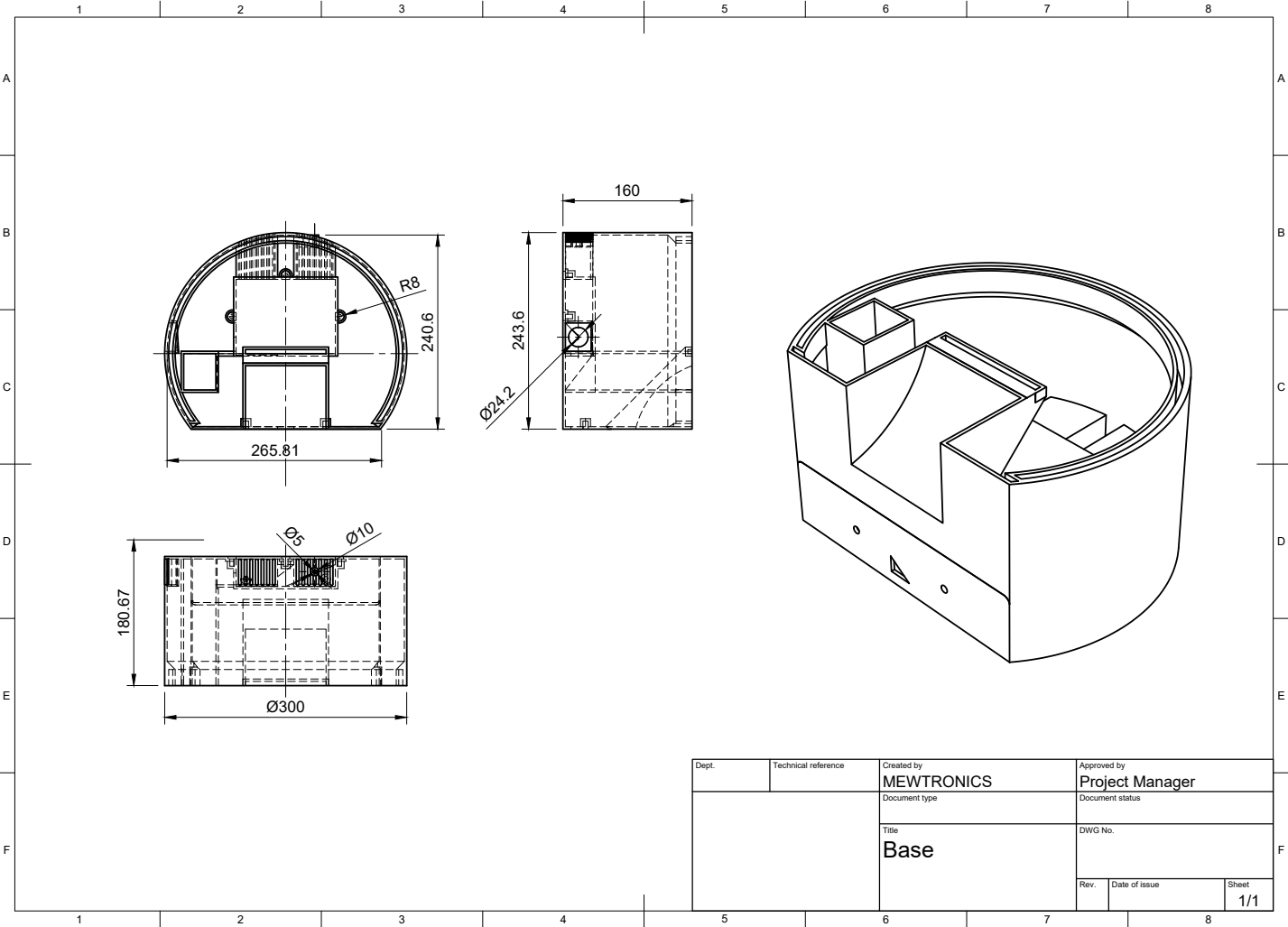
This appendix contains the dimensioned technical drawings of the 3D-printed parts used in the automatic cat feeder prototype.

- BowlBase
- Base
- Bowl
- Drawer
- Storager
- Humidity sensor suport
- Auger Screw
- Gasket
- Central gasket
- Central Part DD
- Central Part DE
- Central Part FD
- Central Part FE
- Central Part Q1
- Central Part Q2
- Bowl plate
- SAI cover
- Top cover
- Hopper

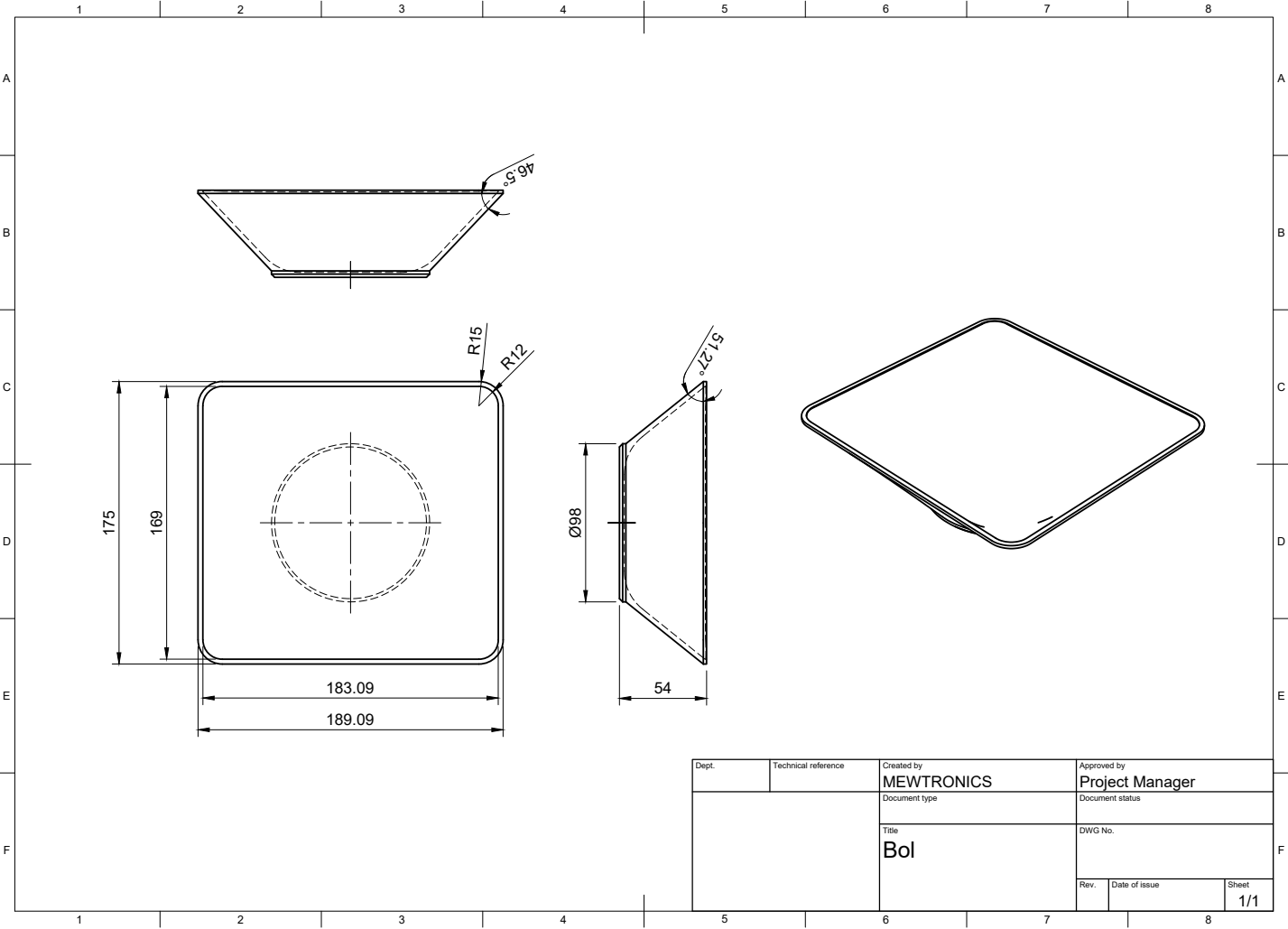
BowlBase



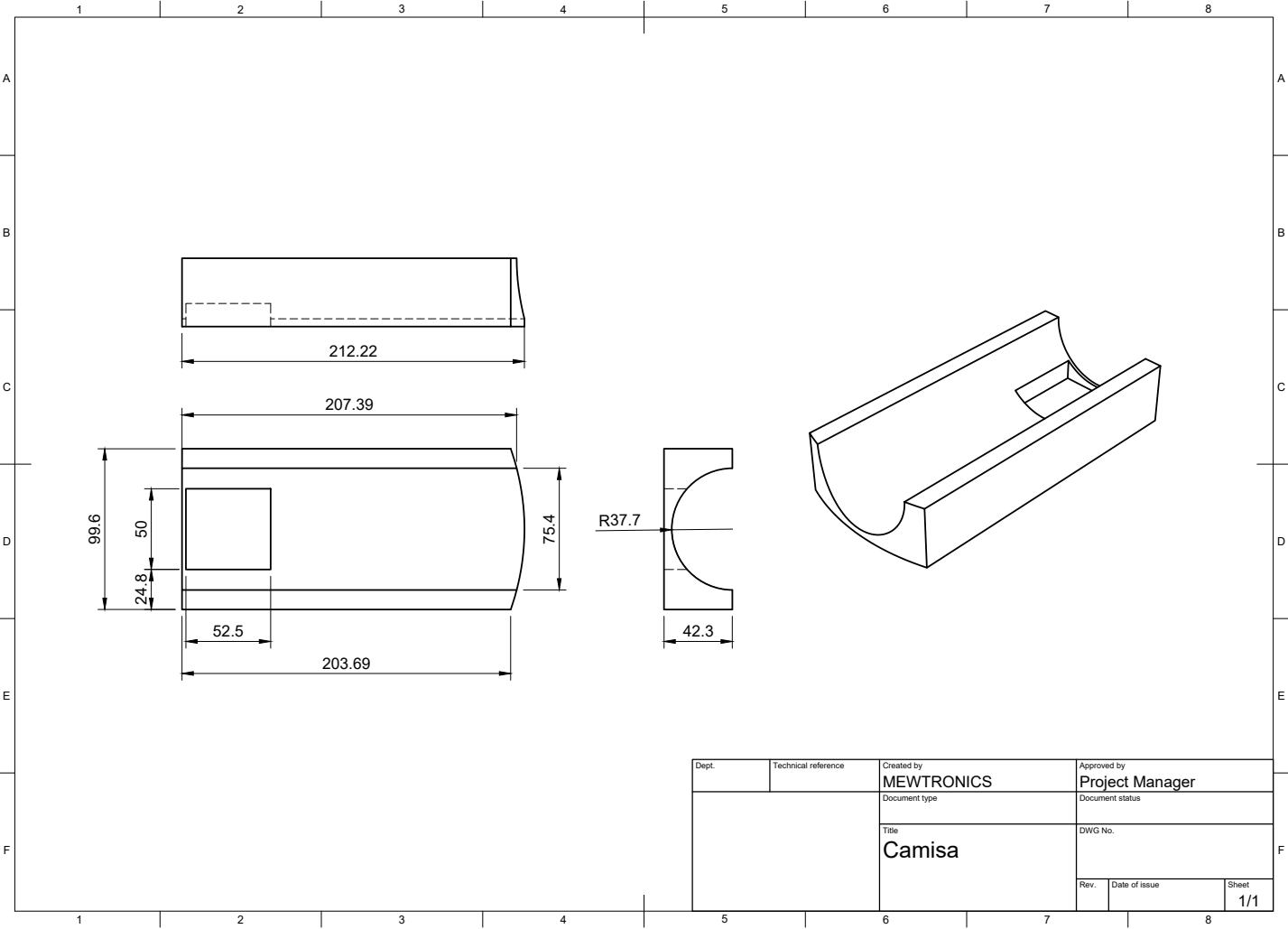
Base



Bowl

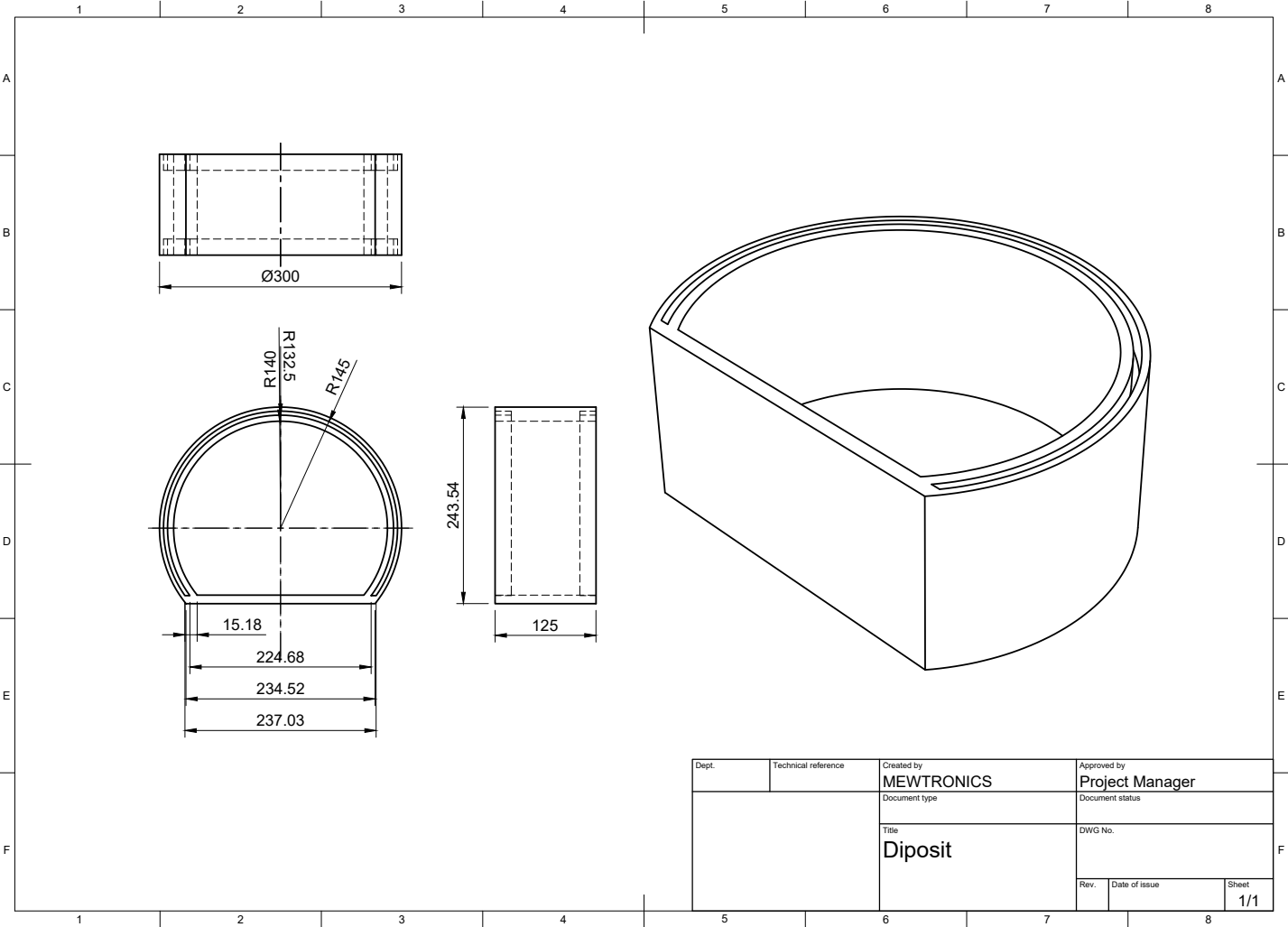


Drawer

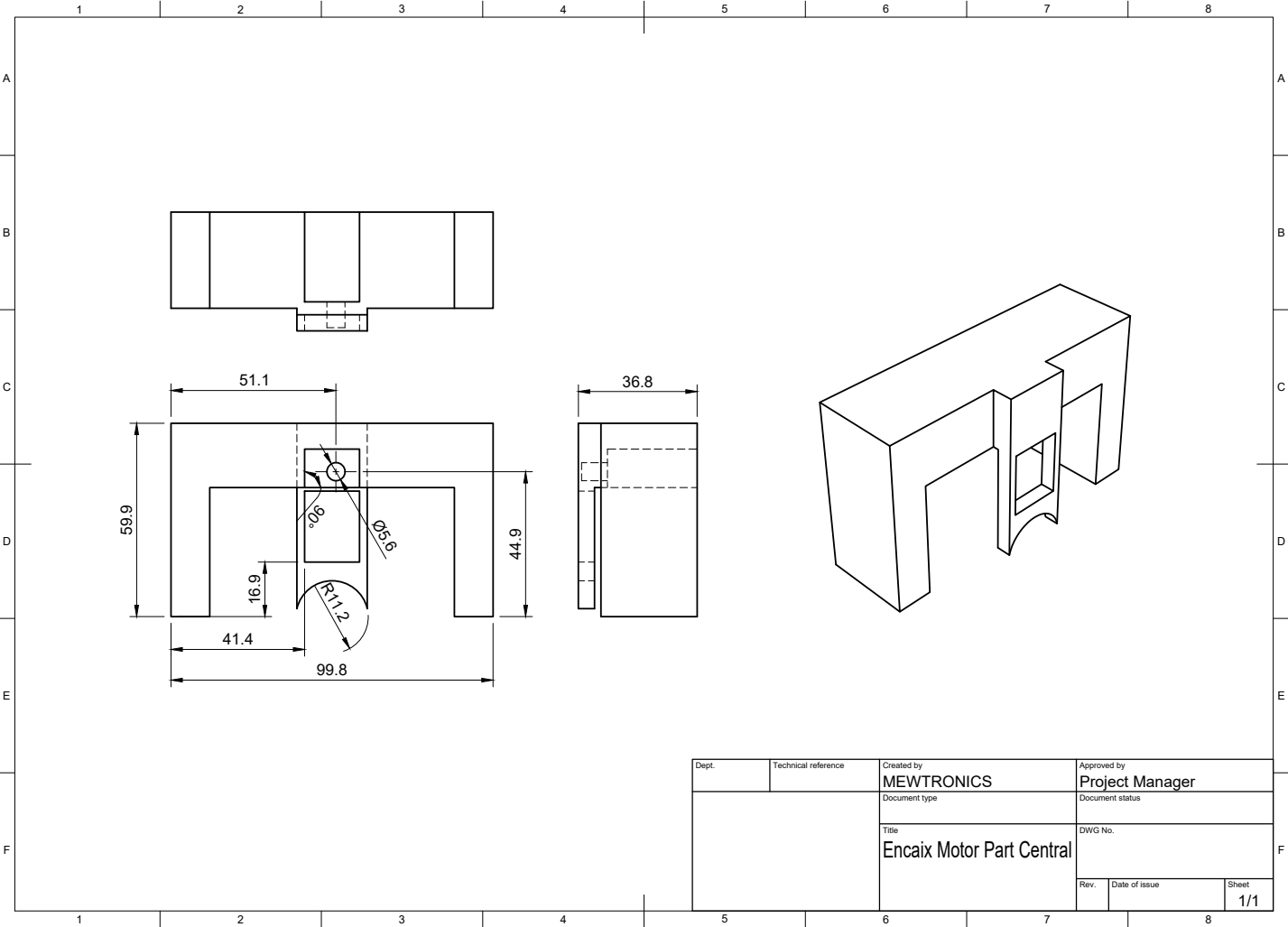




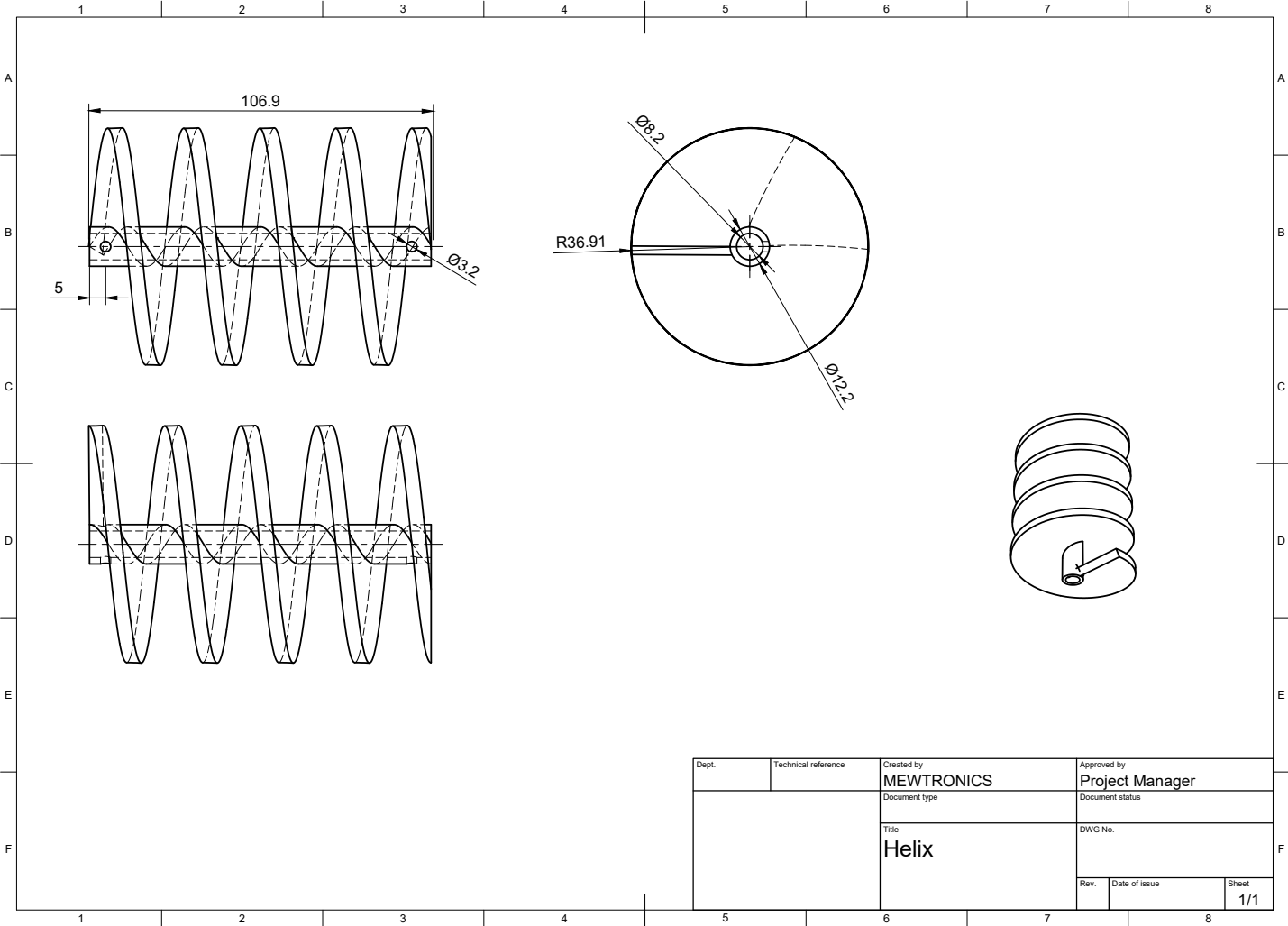
Storager



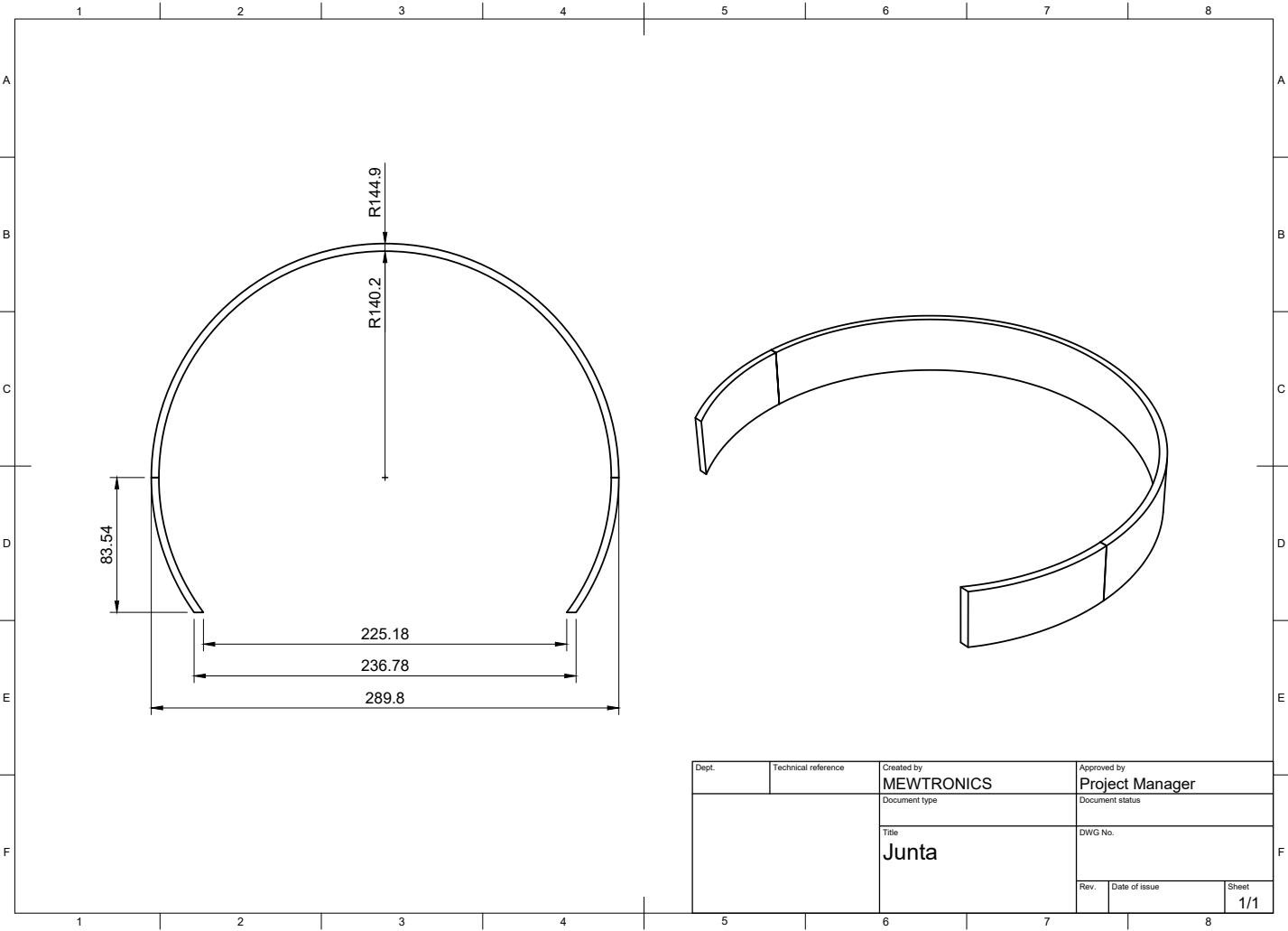
Humidity sensor suport



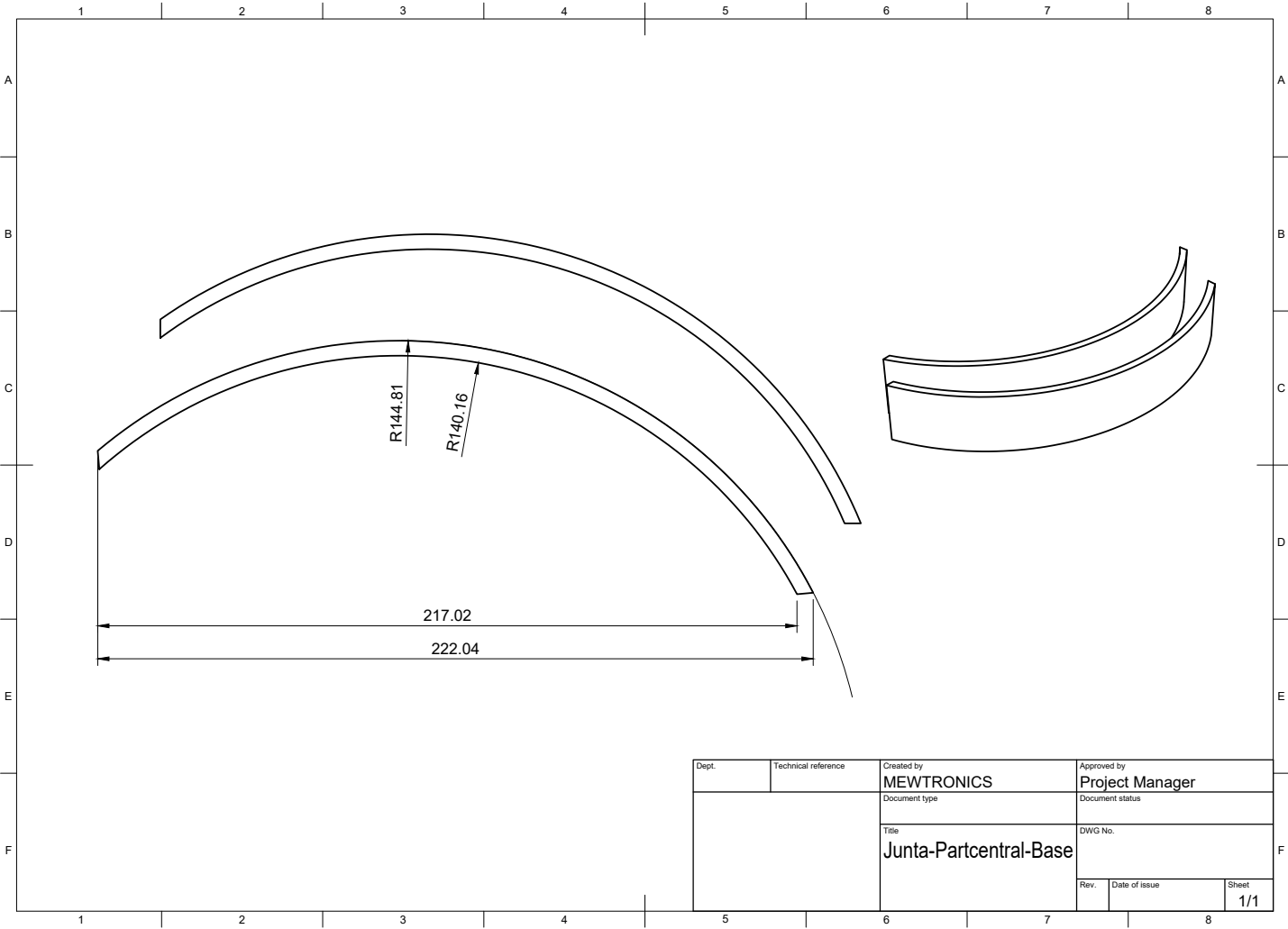
Auger Screw



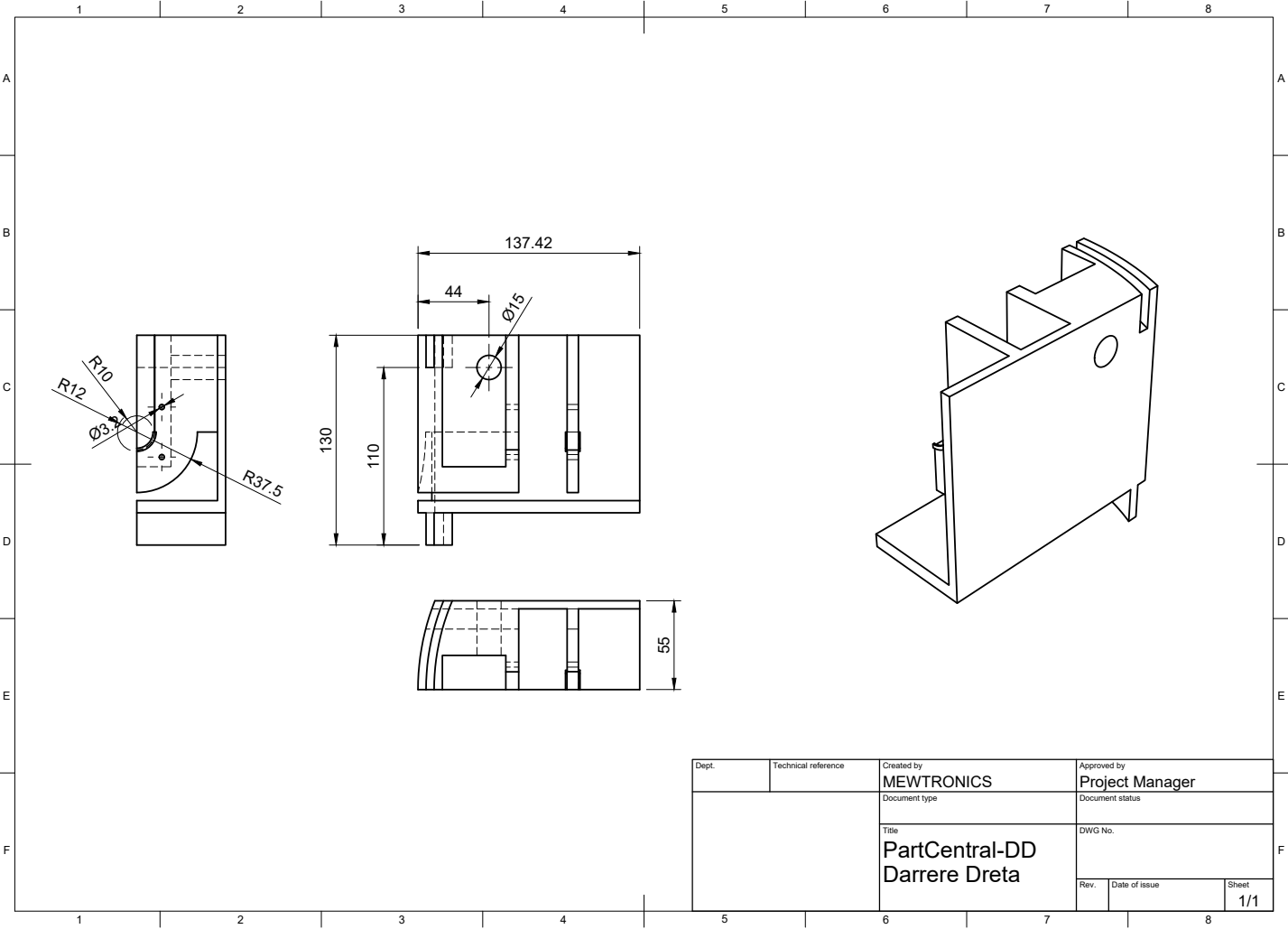
Gasket



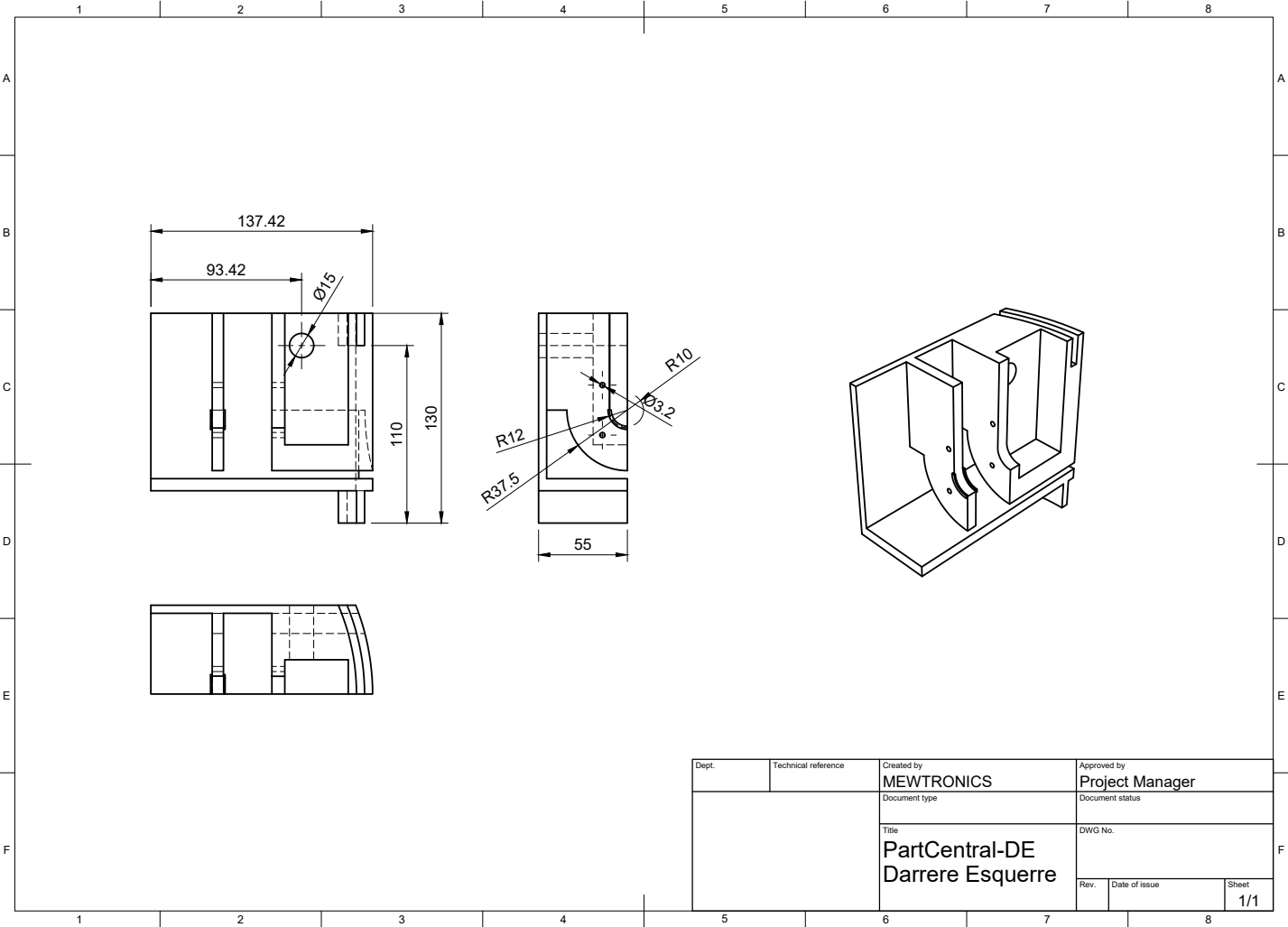
Central gasket



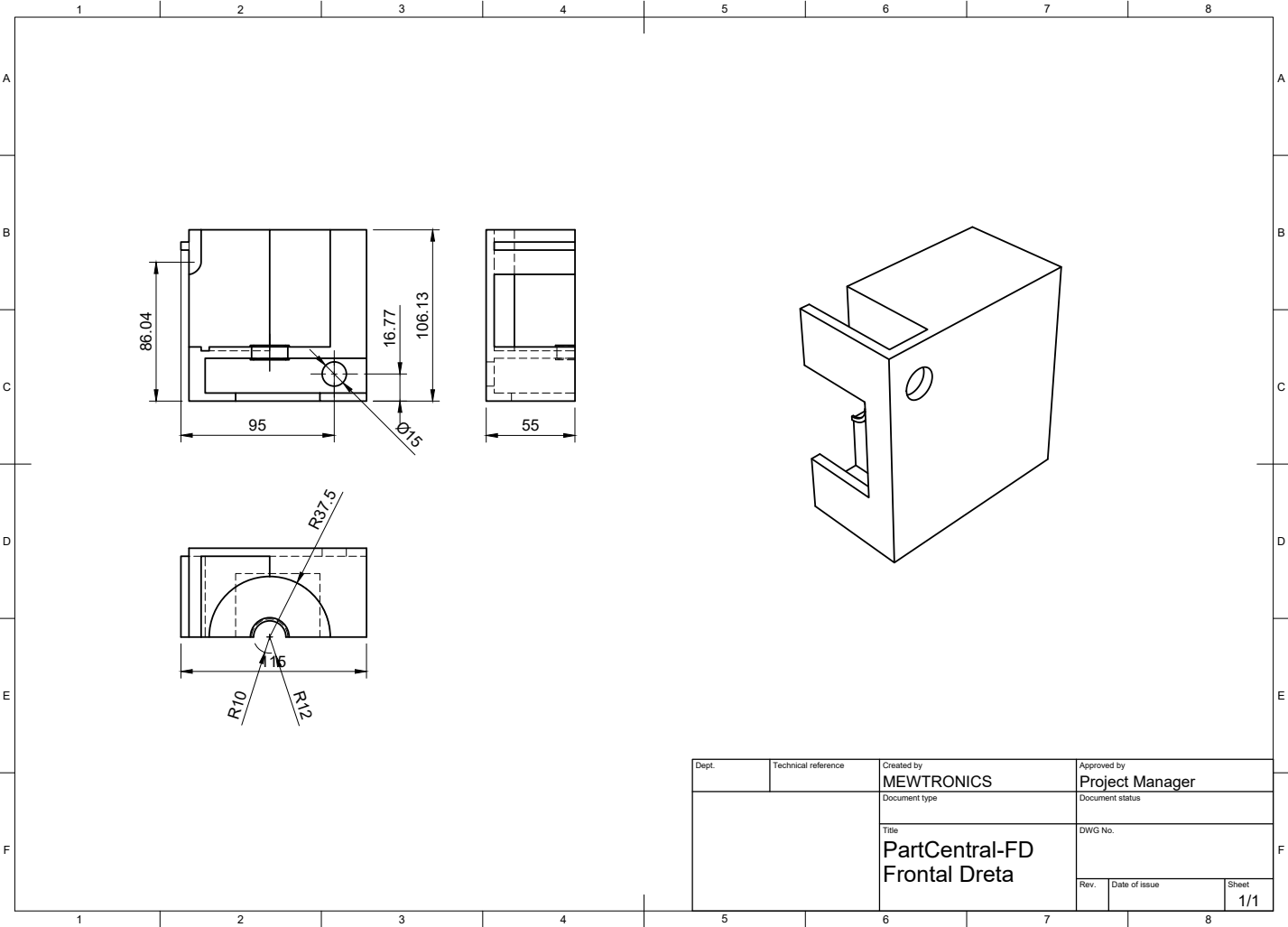
Central Part DD



Central Part DE

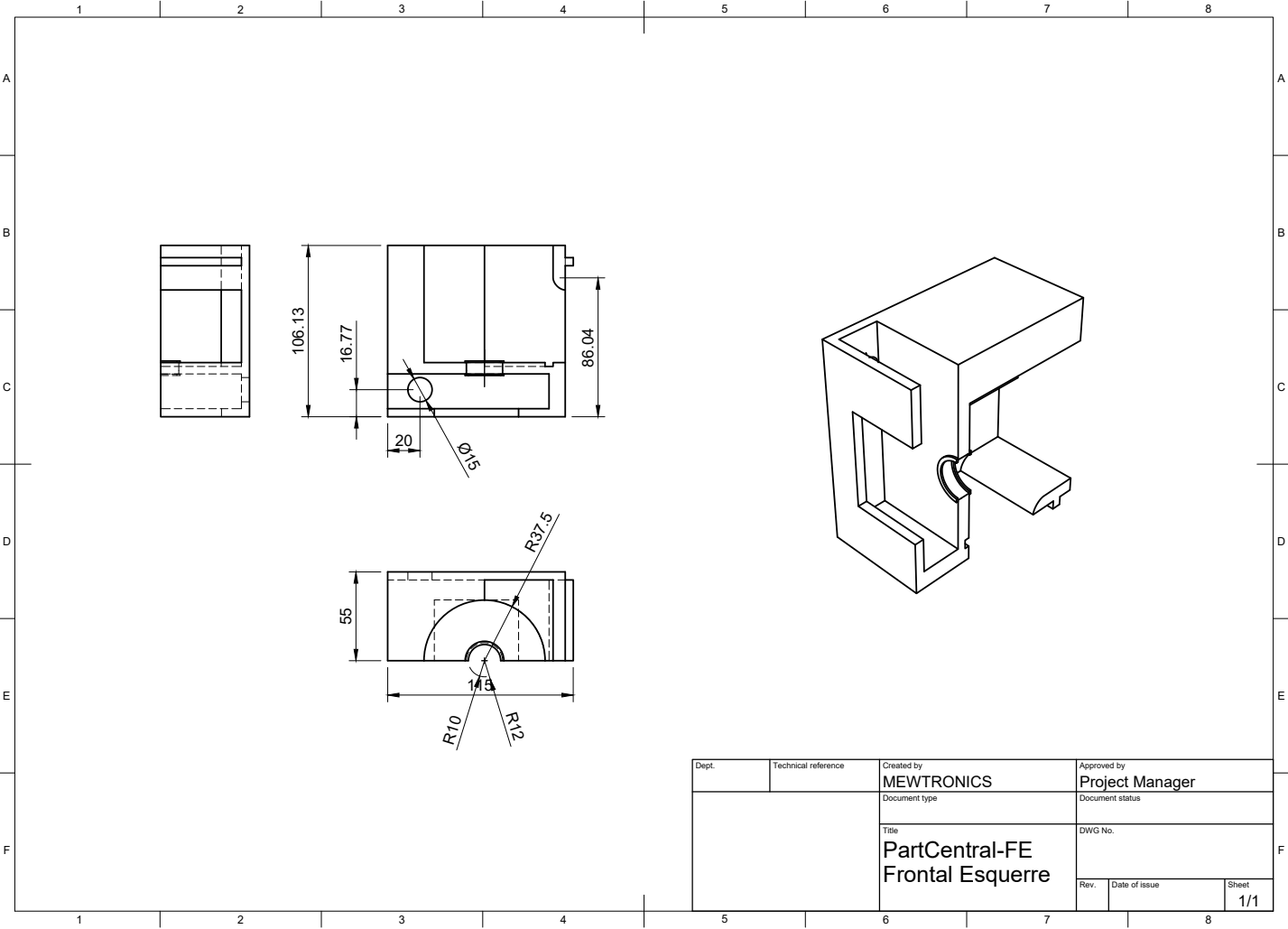


Central Part FD

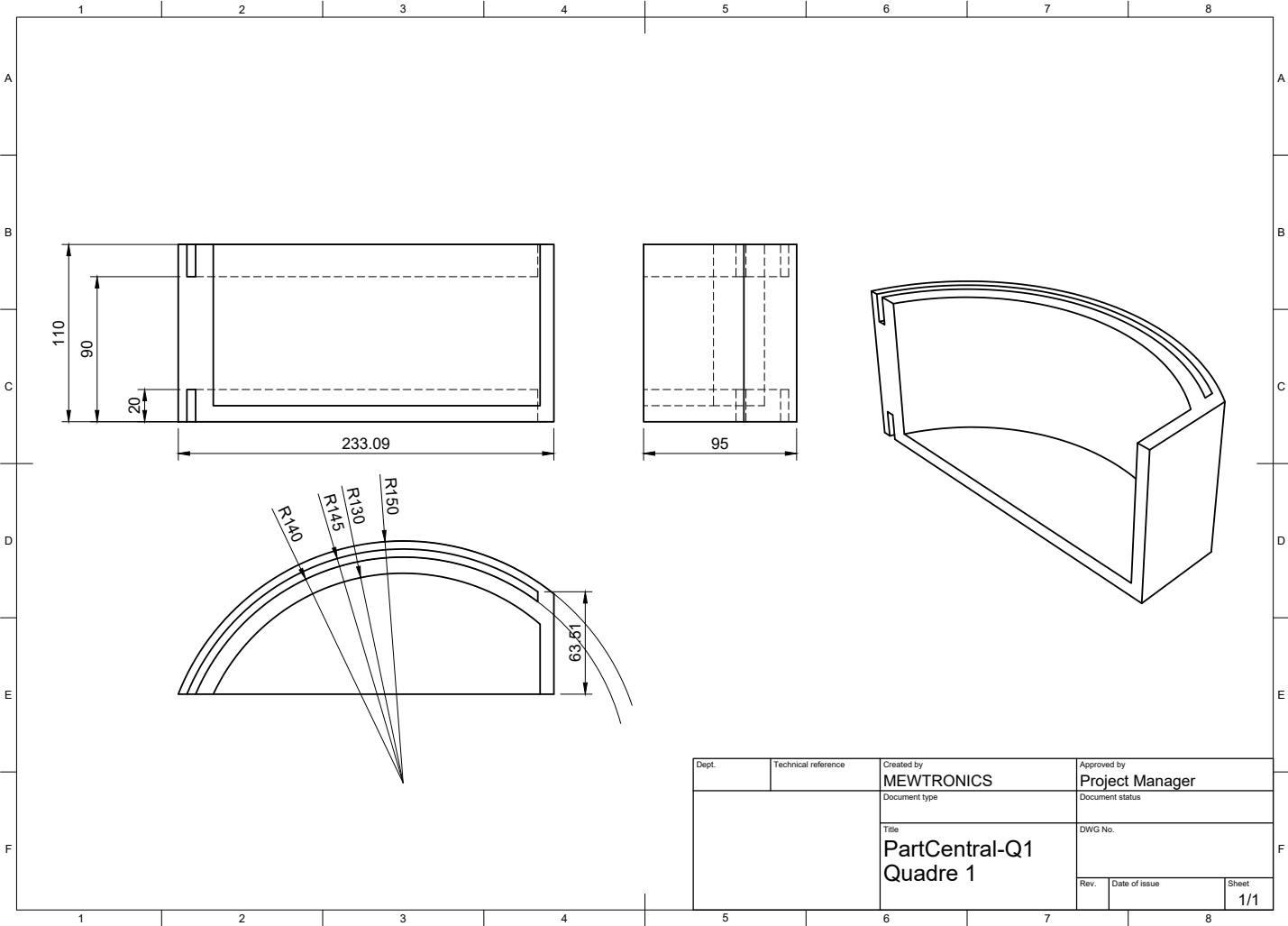




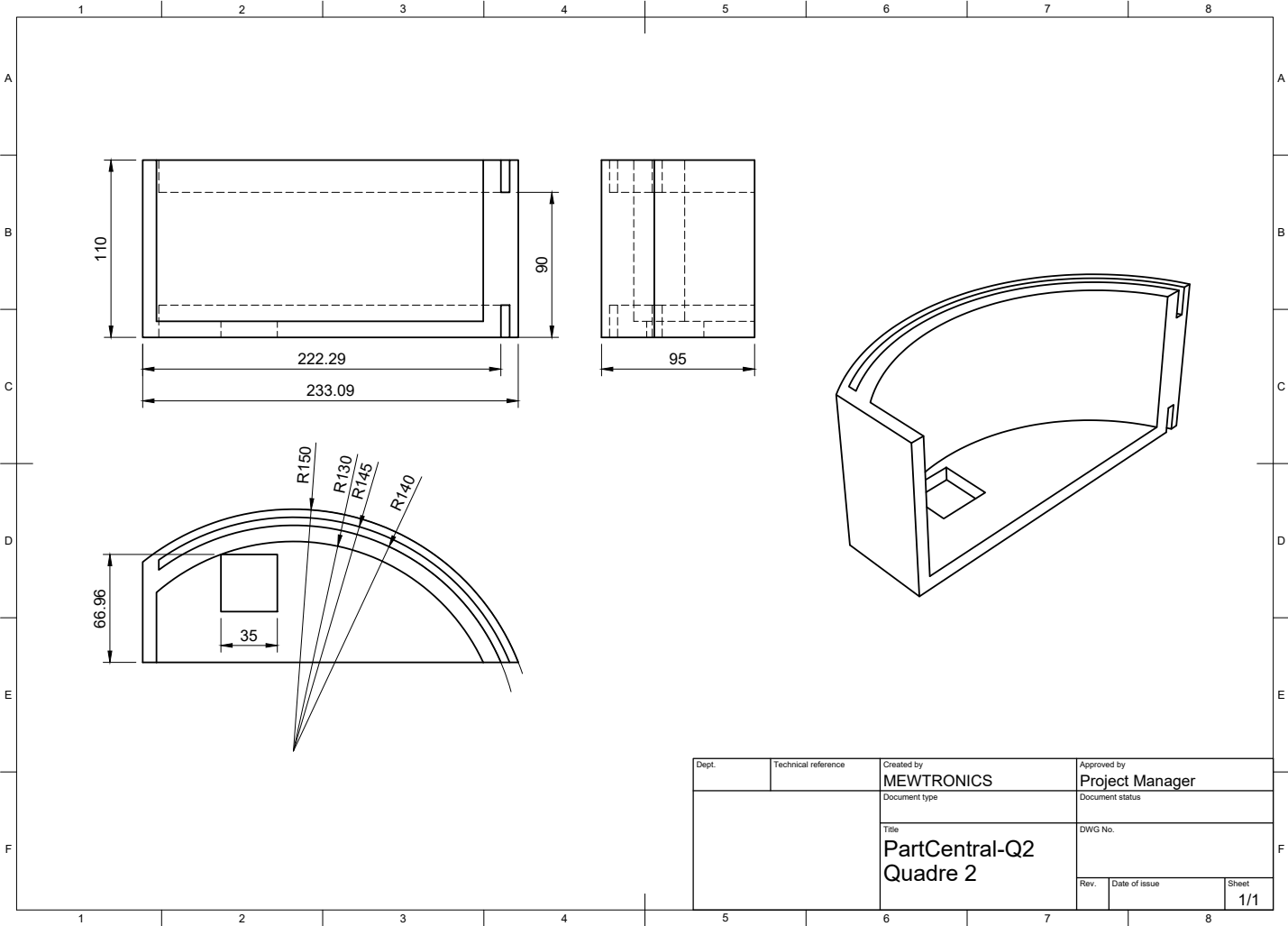
Central Part FE



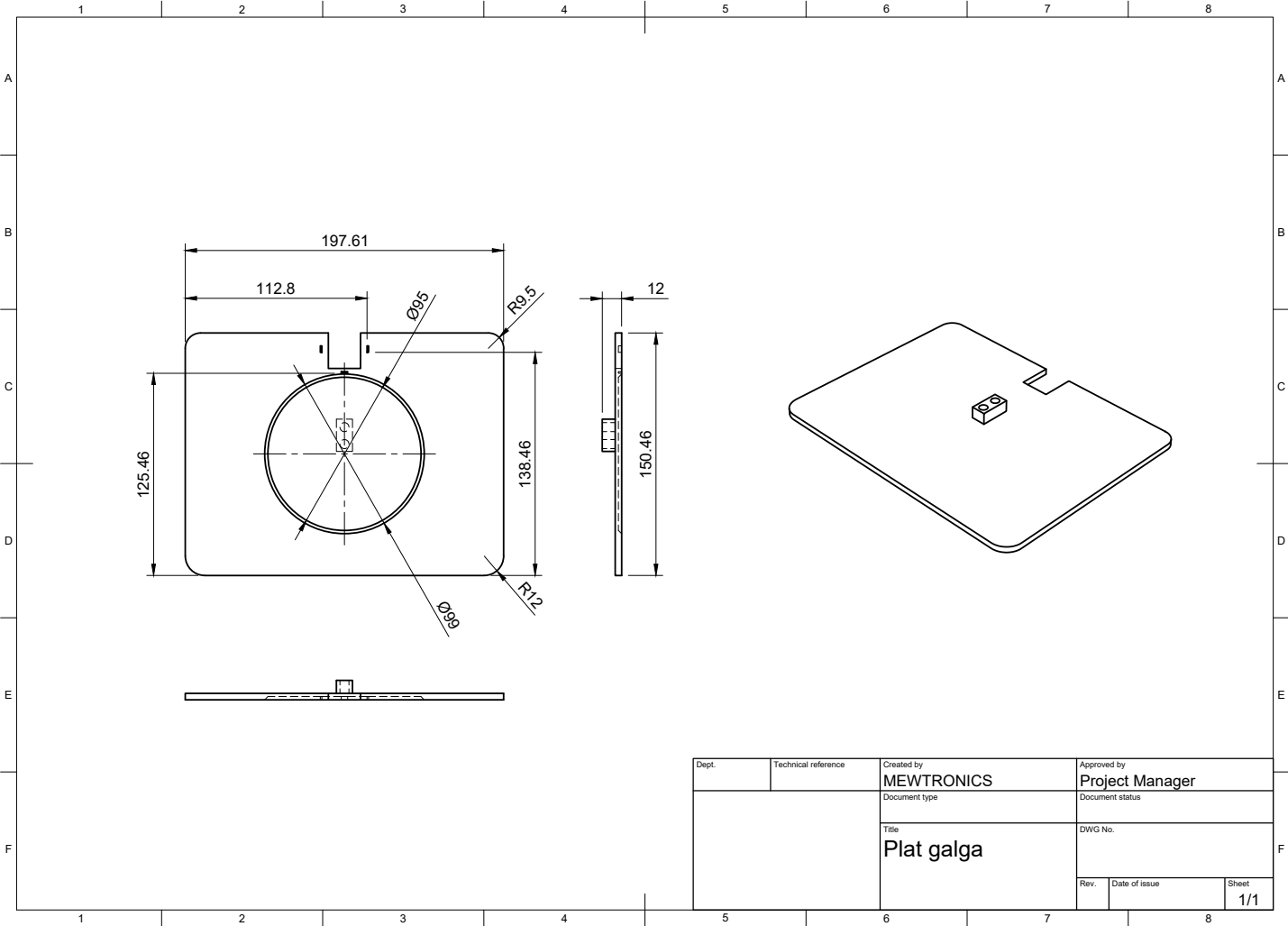
Central Part Q1



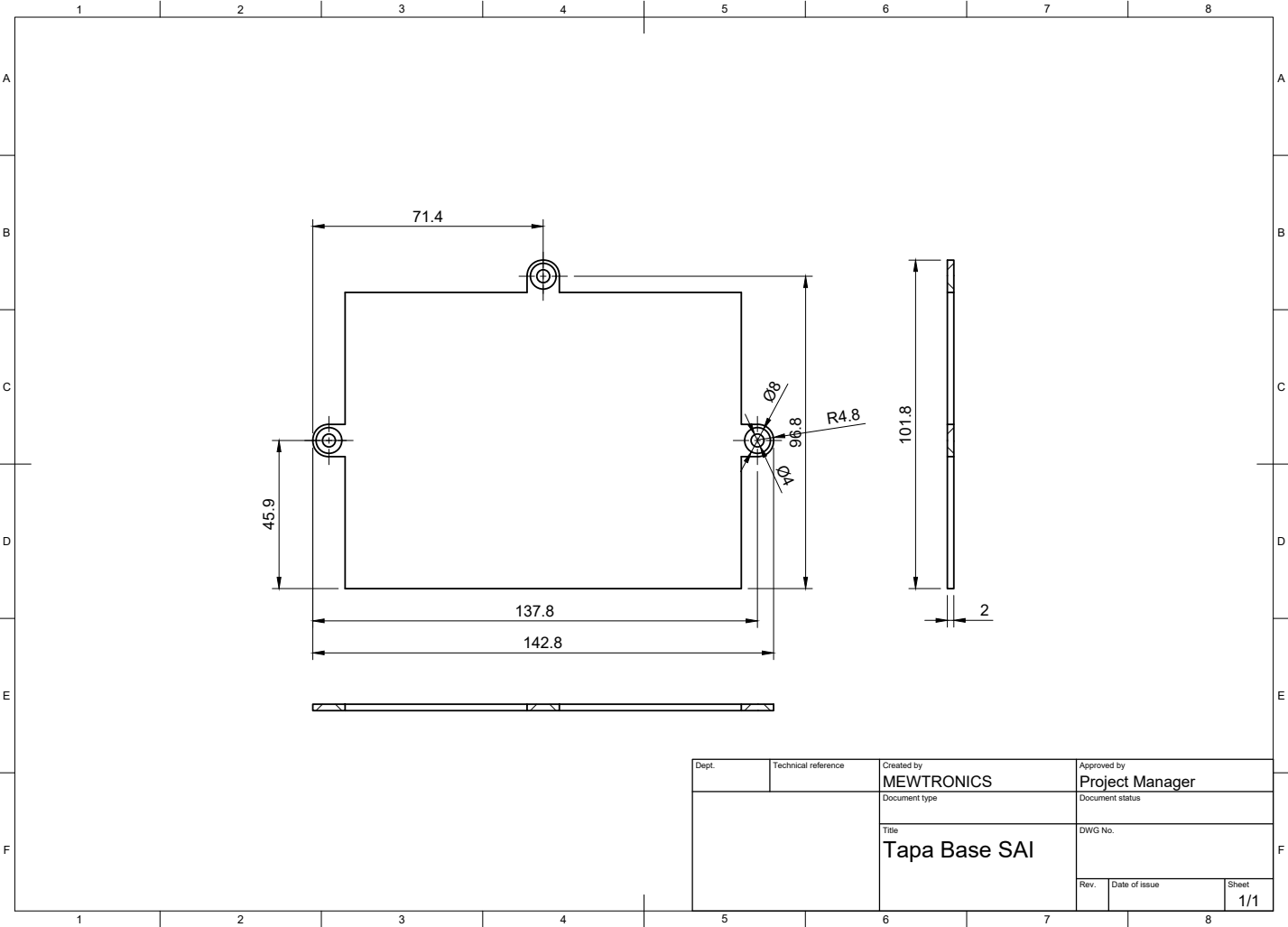
Central Part Q2



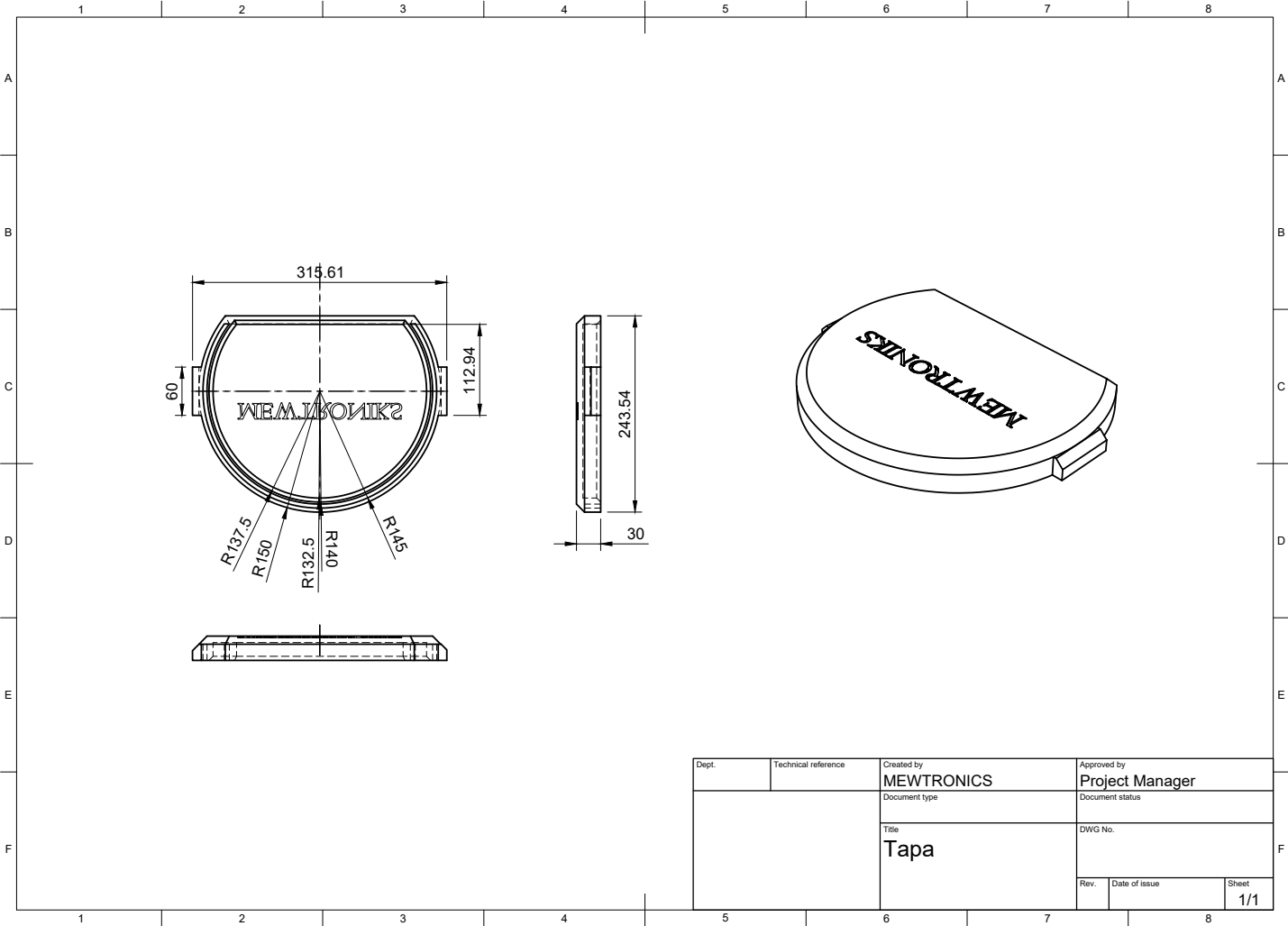
Bowl plate



SAI cover



Top cover



Hopper

